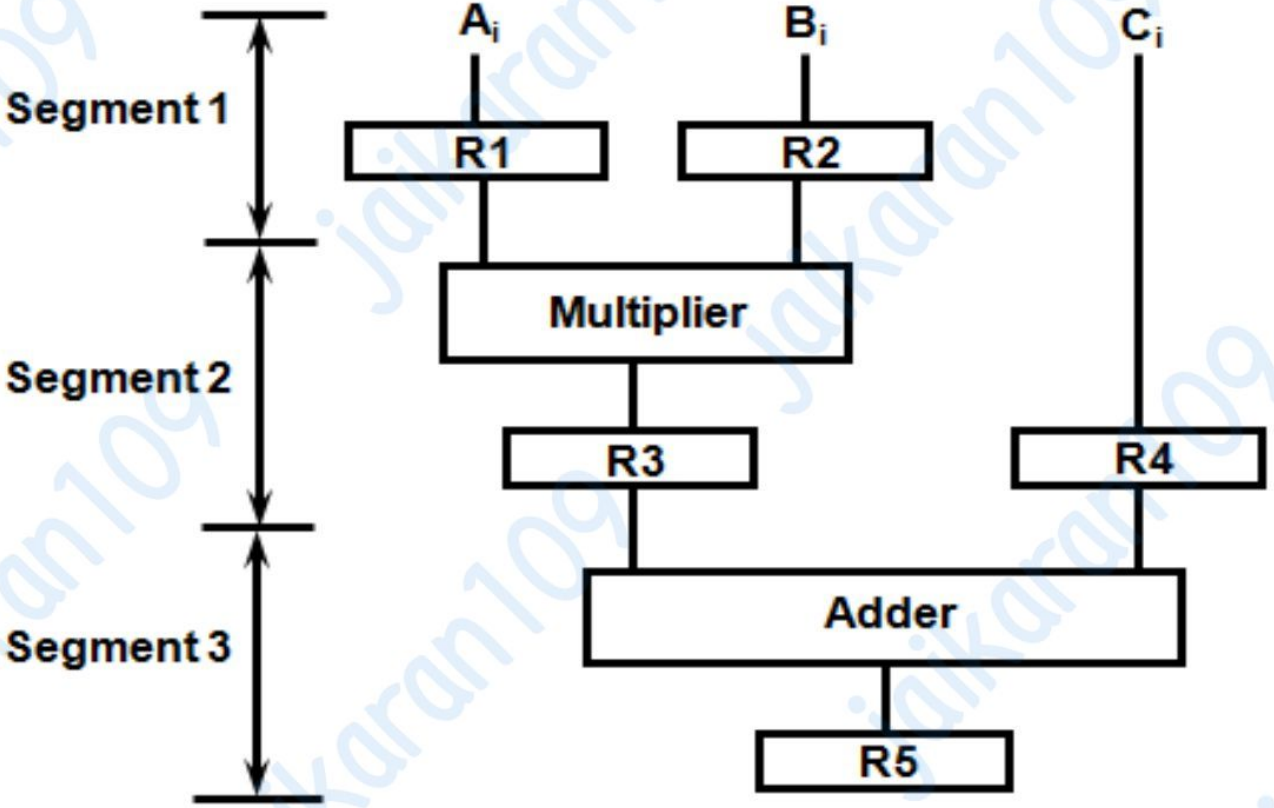
PIPELINING

- Pipelining is a technique of decomposing a sequential process into sub-operations, with each sub-process being executed in a segment that operates concurrently with all other segment.

jaikaran109

Example for Pipeline Processing

$A_i * B_i + C_i$ for $i = 1, 2, 3, \ldots, 7$

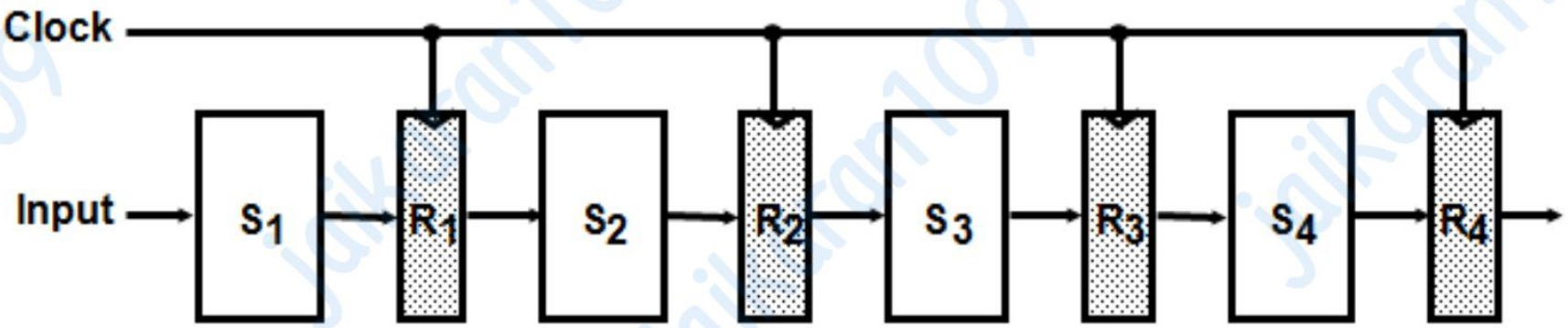


$R1 \leftarrow A_i$, $R2 \leftarrow B_i$	Input A_i and B_i
$R3 \leftarrow R1 * R2$, $R4 \leftarrow C_i$	Multiply and input C_i
$R5 \leftarrow R3 + R4$	Add C_i to product

Operations in each pipeline stage

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A1	B1	---	---	---
2	A2	B2	A1 * B1	C1	---
3	A3	B3	A2 * B2	C2	A1 * B1 + C1
4	A4	B4	A3 * B3	C3	A2 * B2 + C2
5	A5	B5	A4 * B4	C4	A3 * B3 + C3
6	A6	B6	A5 * B5	C5	A4 * B4 + C4
7	A7	B7	A6 * B6	C6	A5 * B5 + C5
8	---	---	A7 * B7	C7	A6 * B6 + C6
9	---	---	---	---	A7 * B7 + C7

General Four-Segment Pipeline



Space-Time Diagram

	1	2	3	4	5	6	7	8	9
Segment 1	T1	T2	T3	T4	T5	T6			
2		T1	T2	T3	T4	T5	T6		
3			T1	T2	T3	T4	T5	T6	
4				T1	T2	T3	T4	T5	T6

Clock cycles

Pipeline Speedup

n: Number of tasks to be performed

Pipelined Machine (k segments)
k-segment pipeline with a clock cycle time of t_p is used to execute n tasks

The first task T_1 requires time = $k*t_p$

The remaining (n-1) tasks emerge from the pipe at the rate of one task per clock cycle = $(n-1)* t_p$

To complete n task with k-segment pipeline= k + (n-1) clock cycles

Conventional Machine (Non-Pipelined)

Time required to complete each task = t_n
Time required to complete the n tasks = $n * t_n$

Speedup

S_k: Speedup

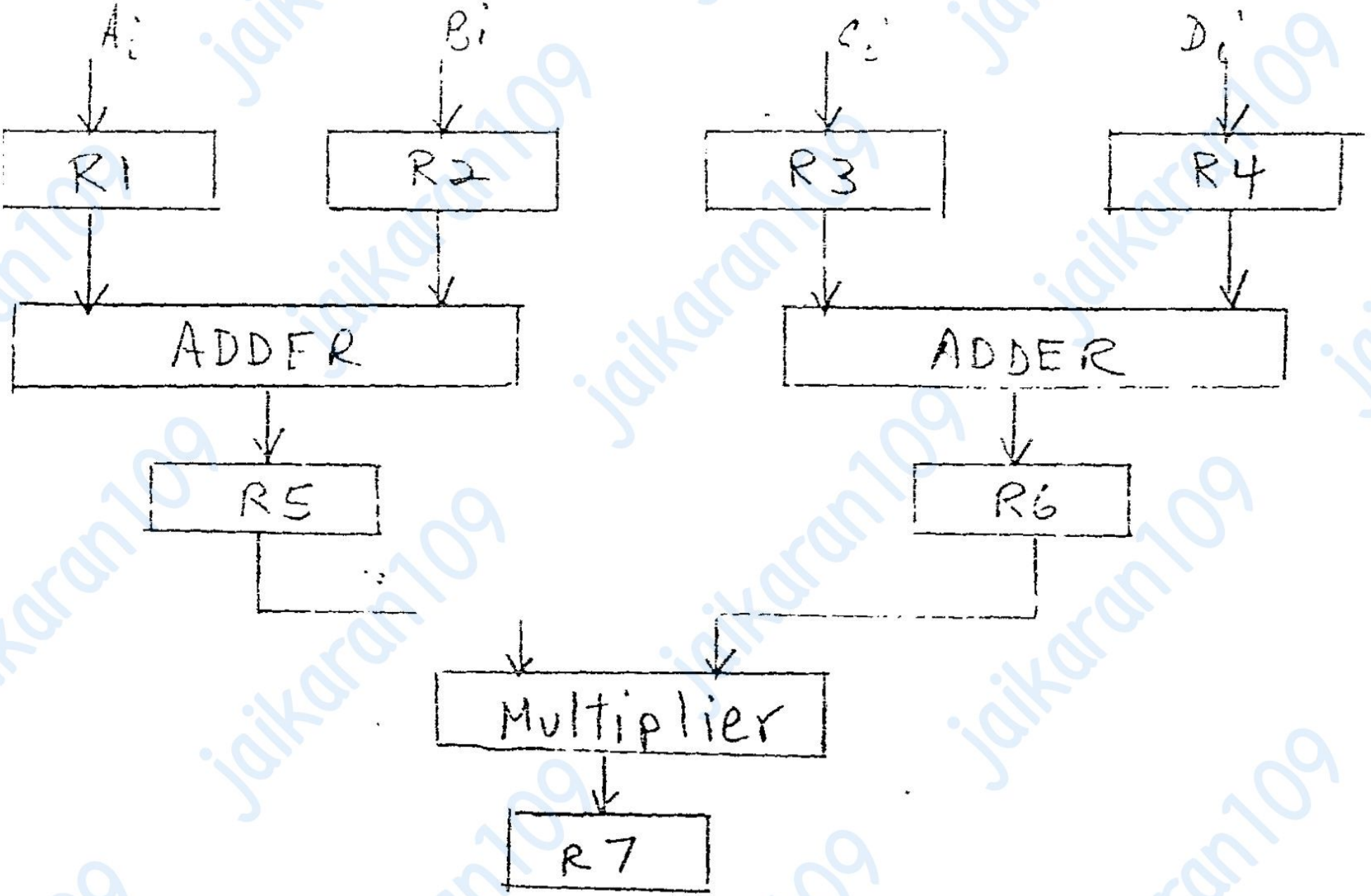
$S_k = n*t_n / (k + n - 1)*t_p$

Q1. In certain scientific computation it is necessary to perform the arithmetic operation (Ai+Bi)*(Ci+Di)

a. Specify the pipeline configuration to carry out this task.
b. List the content of all registers of all the registers in the pipeline for i=1 to 6.

Q2. Draw a space time diagram for a six-segment pipeline showing the time it takes to process eight tasks.

Q3. A non-pipelined system takes 50ns to process a task. The same task can be processed in a six-segment pipeline with a clock cycle of 10ns. Determine the speedup ratio of the pipeline for 100 tasks.


A;
B;
C;
D;
R1
R2
R3
R4
ADDER
ADDER
R5
R6
Multiplier
R7

Segment	1	2	3	4	5	6	7	8	9	10	11	12	13
1	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8					
2		T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8				
3			T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8			
4				T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8		
5					T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	
6						T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8

$(k+n-1)t_p = 6+8-1 = 13$ cycles

$t_n = 50$ ns

$k = 6$

$t_p = 10$ ns

$n = 100$

$$S = \frac{n t_n}{(k + n - 1) t_p} = \frac{100 \times 50}{(6 + 99) \times 10} = 4.76$$

$$S_{max} = \frac{t_n}{t_p} = \frac{50}{10} = 5$$

Problem

- Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- 1. Pipeline cycle time
- 2. Non-pipeline execution time
- 3. Speed up ratio
- 4. Pipeline time for 1000 tasks
- 5. Sequential time for 1000 tasks
- 6. Throughput

jaikaran109

- **Solution-**
- Given-
- Four stage pipeline is used
- Delay of stages = 60, 50, 90 and 80 ns
- Latch delay or delay due to each register = 10 ns

- **Part-01:**
- **Pipeline Cycle Time-**
- Cycle time
- = Maximum delay due to any stage + Delay due to its register
- = Max { 60, 50, 90, 80 } + 10 ns
- = 90 ns + 10 ns
- = 100 ns

- **Part-02: Non-Pipeline Execution Time-**
- Non-pipeline execution time for one instruction
- = 60 ns + 50 ns + 90 ns + 80 ns
- = 280 ns
- **Part-03: Speed Up Ratio-**
- Speed up
- = Non-pipeline execution time / Pipeline execution time
- = 280 ns / Cycle time
- = 280 ns / 100 ns
- = 2.8

jaikaran109

- **Part-04: Pipeline Time For 1000 Tasks-**
- Pipeline time for 1000 tasks
- = Time taken for 1st task + Time taken for remaining 999 tasks
- = 1 x 4 clock cycles + 999 x 1 clock cycle
- = 4 x cycle time + 999 x cycle time
- = 4 x 100 ns + 999 x 100 ns
- = 400 ns + 99900 ns
- = 100300 ns

- **Part-05: Sequential Time For 1000 Tasks-**
- Non-pipeline time for 1000 tasks
- = 1000 x Time taken for one task
- = 1000 x 280 ns
- = 280000 ns

- **Part-06: Throughput-**
- Throughput for pipelined execution
- = Number of instructions executed per unit time
- = 1000 tasks / 100300 ns

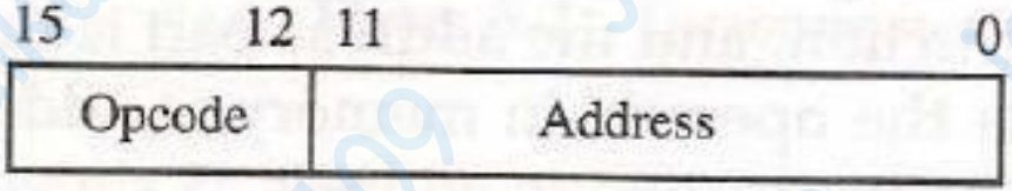
BASIC COMPUTER ORGANIZATION AND DESIGN

- Instruction Codes
- Computer Registers
- Computer Instructions
- Timing And Control
- Instruction Cycle
- Register – Reference Instructions
- Memory – Reference Instructions
- Input – Output And Interrupt

jaikaran109

Instruction Codes:

- Program: set of instructions that specify the operations, operands, and the sequence by which processing has to occur.
- Instruction: a binary code that specifies a sequence of micro-operations for the computer.
- The computer reads each instruction from memory and places it in a control register. The control then interprets the binary code of the instruction and proceeds to execute it by issuing a sequence of micro-operations. – Instruction Cycle
- Instruction Code: group of bits that instruct the computer to perform specific operation.
- Instruction code is usually divided into two parts: Opcode and address(operand)



Instruction format

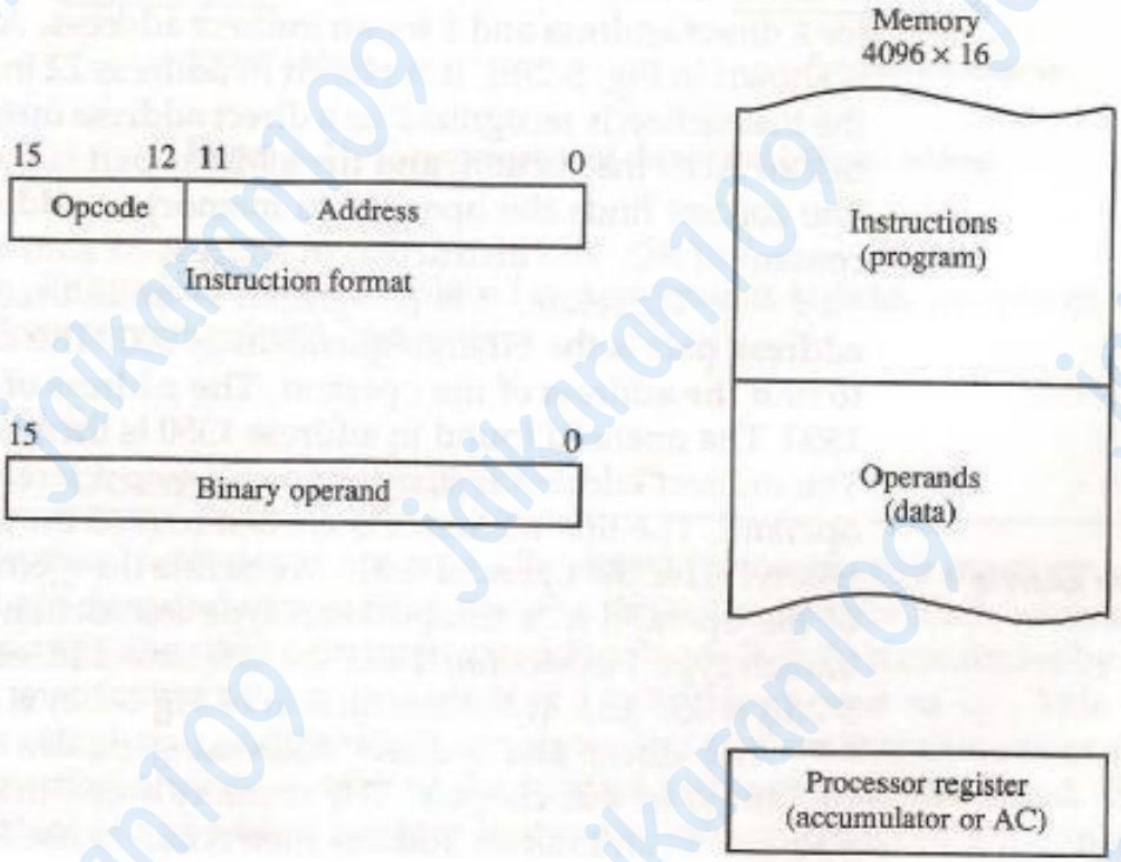
Operation Code (opcode):

- group of bits that define the operation
- Eg: add, subtract, multiply, shift, complement.

Address (operand):

- specifies the location of operands (registers or memory words)
- Memory words are specified by their address
- Registers are specified by their k-bit binary code

Figure 5-1 Stored program organization.



The ability to store and execute instructions is the most important property of a general-purpose computer. That type of stored program concept is called stored program organization.

- Accumulator (AC):
- Computers that have a single-processor register usually assign to it the name accumulator and label it AC.
- The operation is performed with the memory operand and the content of AC.

- **Addressing of Operand:**
- Sometimes convenient to use the address bits of an instruction code not as an address but as the actual operand.
- When the second part of an instruction code specifies an operand, the instruction is said to have an immediate operand.
- When the second part specifies the address of an operand, the instruction is said to have a direct address.
- When second part of the instruction designate an address of a memory word in which the address of the operand is found such instruction have indirect address.

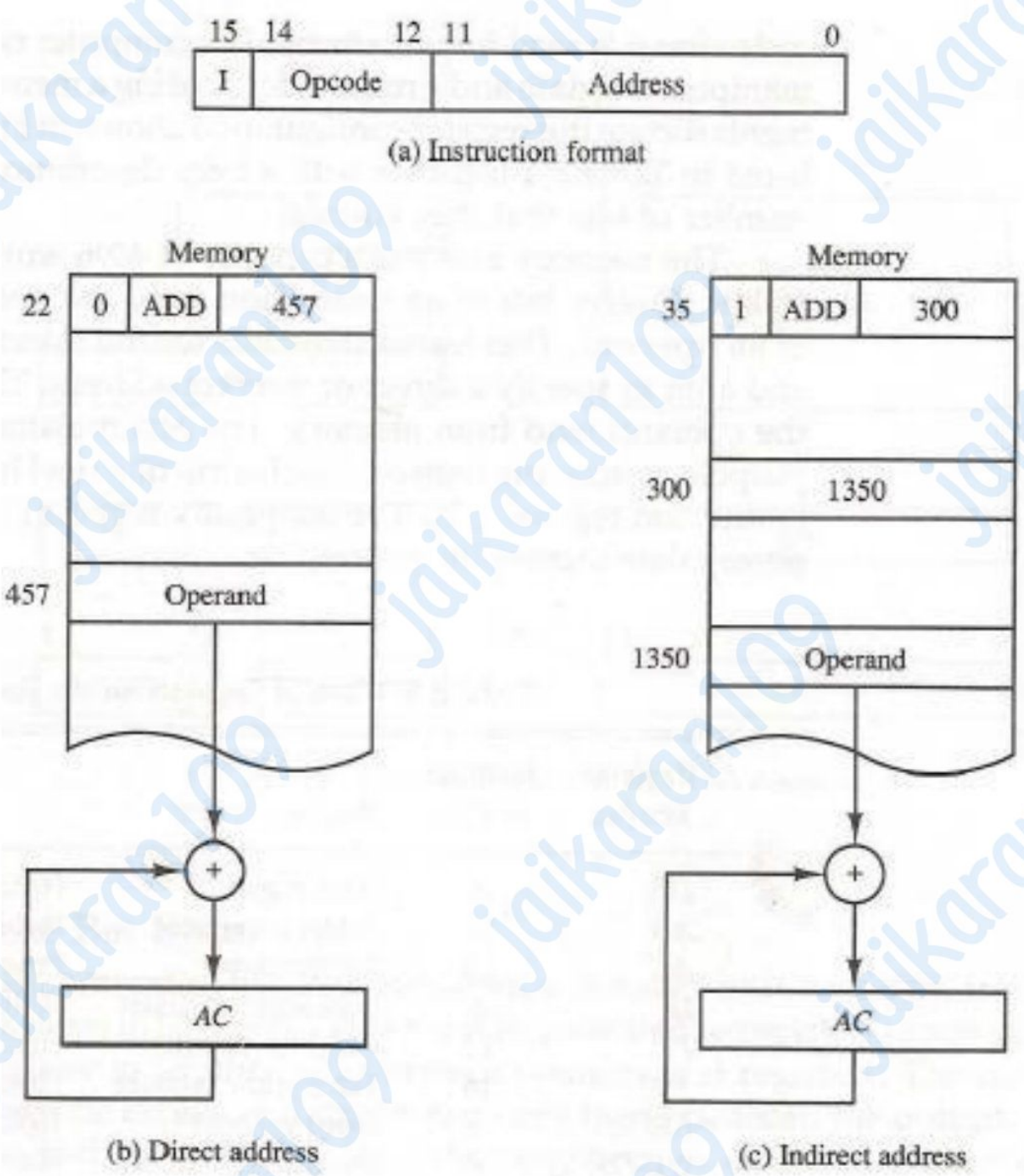


Figure 5-2 Demonstration of direct and indirect address.

- **A direct address instruction is shown in Fig. 5-2(b).**
- It is placed in address 22 in memory. The I bit is 0, so the instruction is recognized as a direct address instruction. The opcode specifies an ADD instruction, and the address part is the binary equivalent of 457.
- The control finds the operand in memory at address 457 and adds it to the content of AC.
- The instruction in address 35 shown in Fig. 5-2(c) has a mode bit I = 1.
- Therefore, it is recognized as an indirect address instruction.
- The address part is the binary equivalent of 300. The control goes to address 300 to find the address
- of the operand. The address of the operand in this case is 1350.

- The operand found in address 1350 is then added to the content of AC.
- The effective address to be the address of the operand in a computation-type instruction or the
- target address in a branch-type instruction.
- Thus the effective address in the instruction of Fig. 5-2(b) is 457 and in the instruction of Fig 5-2(c) is 1350.

Computer Registers:

TABLE 5-1 List of Registers for the Basic Computer

Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

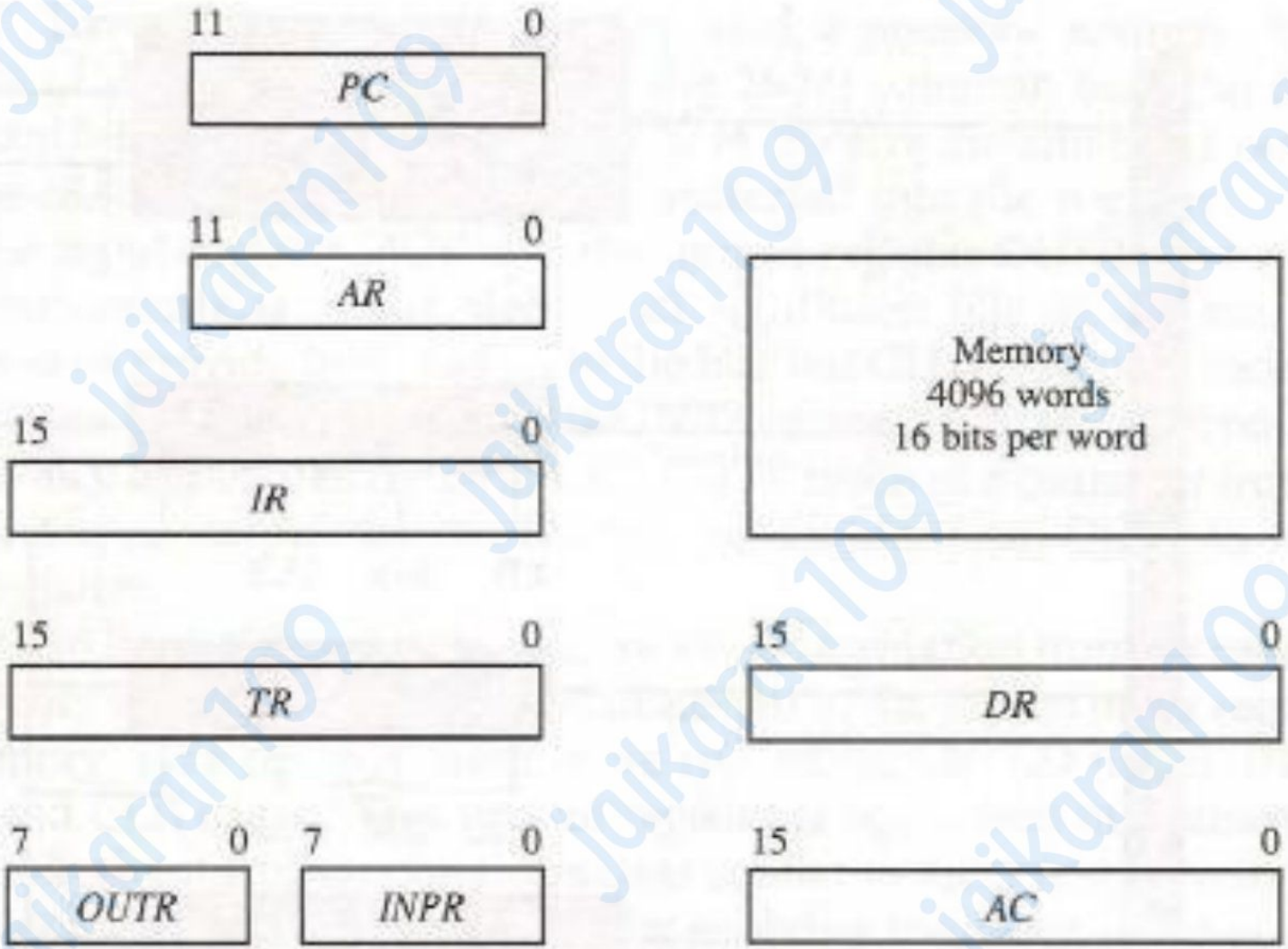


Figure 5-3 Basic computer registers and memory.

- The data register (DR) holds the operand read from memory.
- The accumulator (AC) register is a general purpose processing register.
- The instruction read from memory is placed in the instruction register (IR).
- The temporary register (TR) is used for holding temporary data during the processing.
- The memory address register (AR) has 12 bits since this is the width of a memory address.
- The program counter (PC) also has 12 bits and it holds the address of the next instruction to be read from memory after the current instruction is executed.
- Two registers are used for input and output.
- The input register (INPR) receives an 8-bit character from an input device.
- The output register (OUTR) holds an 8-bit character for an output device.

Common Bus System:

- The basic computer has eight registers, a memory unit, and a control unit .
- Paths must be provided to transfer information from one register to another and between memory and registers.
- A more efficient scheme for transferring information in a system with many registers is to use a common bus.
- The connection of the registers and memory of the basic computer to a common bus system is shown in Fig. 5-4.
- The outputs of seven registers and memory are connected to the common bus.

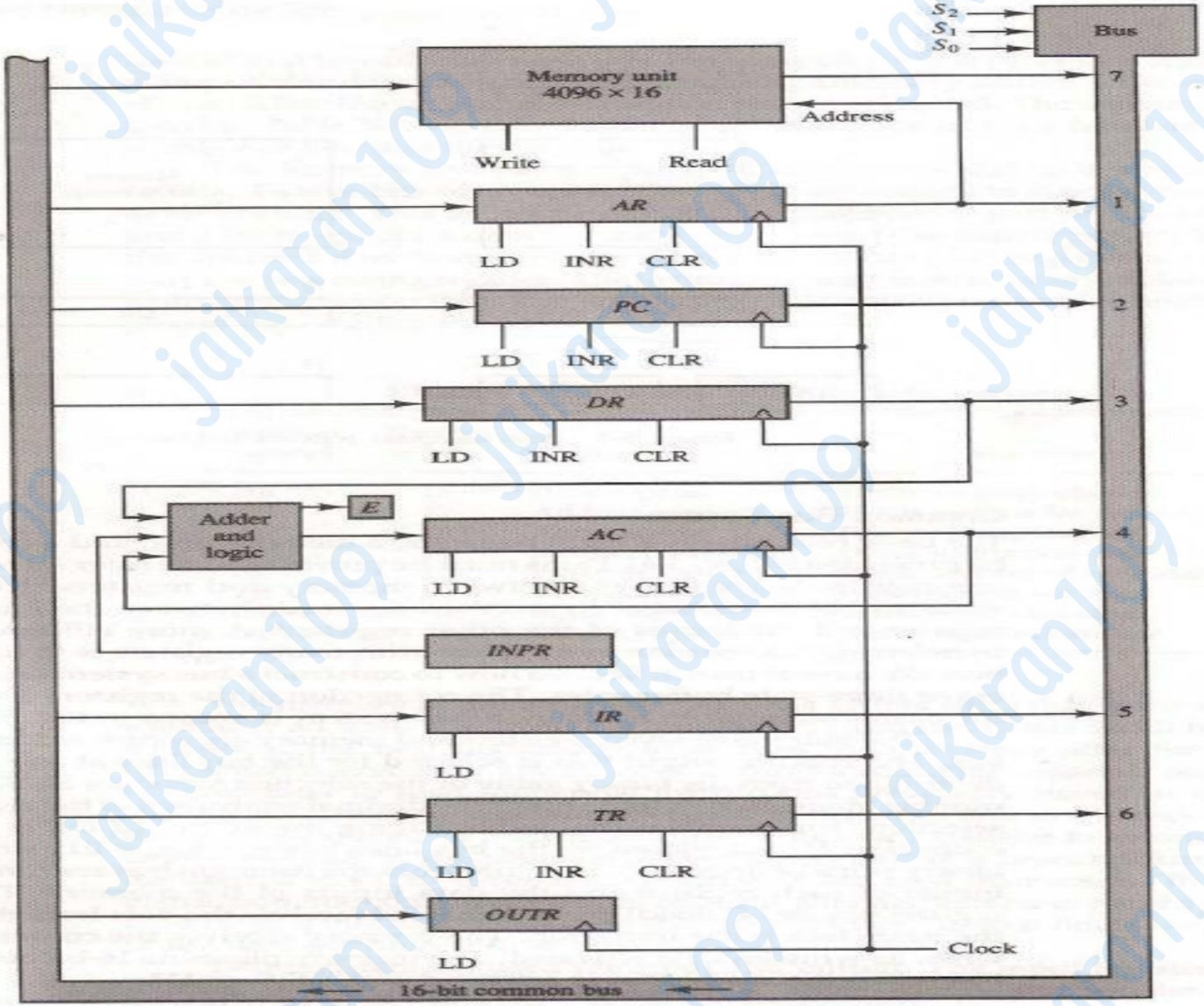


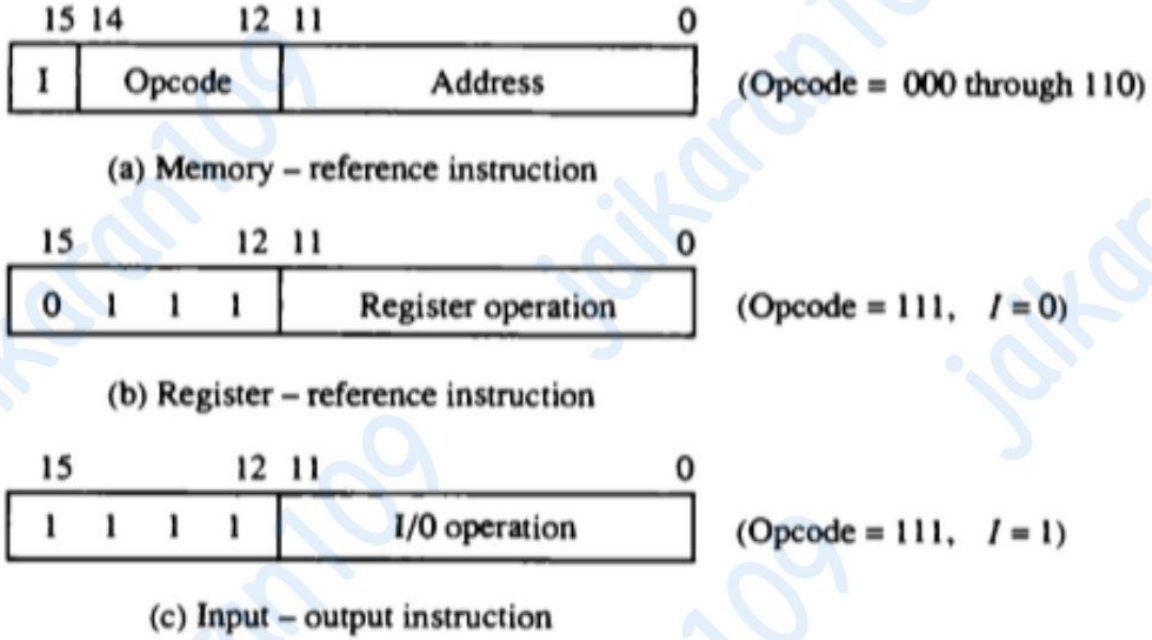
Figure 5-4 Basic computer registers connected to a common bus.

- The number along each output shows the decimal equivalent of the required binary selection.
- For example, the number along the output of DR is 3. The 16-bit outputs of DR are placed on the bus lines when $S_2S_1S_0 = 011$.
- Five registers have three control inputs: LD (load), INR (increment), and CLR (clear).
- The result of an addition is transferred to AC and the end carry-out of the addition is transferred to flip-flop E (extended AC bit).
- A third set of 8-bit inputs come from the input register INPR.
- The content of any register can be applied onto the bus and an operation can be performed in the adder and logic circuit during the same clock cycle.

Computer Instructions

- The basic computer has three instruction code formats.
- Each format has 16 bits.
- The operation code(op-code) part of the instruction contains three bits and the meaning of remaining 13 bits depends upon the op-code encountered.

Basic computer instruction formats.



Computer Instruction Set

Basic Computer Instructions

Symbol	Hexadecimal code $I = 0$	Hexadecimal code $I = 1$	Description
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC* in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear *AC*
CLE	7400		Clear *E*
CMA	7200		Complement *AC*
CME	7100		Complement *E*
CIR	7080		Circulate right *AC* and *E*
CIL	7040		Circulate left *AC* and *E*
INC	7020		Increment *AC*
SPA	7010		Skip next instruction if *AC* positive
SNA	7008		Skip next instruction if *AC* negative
SZA	7004		Skip next instruction if *AC* zero
SZE	7002		Skip next instruction if *E* is 0
HLT	7001		Halt computer
INP	F800		Input character to *AC*
OUT	F400		Output character from *AC*
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

TIMING and CONTROL UNIT

jaikaran109

INTRODUCTION

- The Control Unit was historically defined as one distinct part of the 1946 reference model of Von Neumann architecture.

- The function of control unit is to generate relevant timing and control signals to all operations in the computer.

- It controls the flow of data between the processor and memory and peripherals.

- The examples of devices that require a control unit are CPUs and graphics processing units (GPUs).

FUNCTIONS

- The control unit must communicate with both the arithmetic logic unit (ALU) and main memory.
- The control unit instructs the arithmetic logic unit that which logical or arithmetic operation is to be performed.
- The control unit co-ordinates the activities of the other two units as well as all peripherals and auxiliary storage devices linked to the computer.

jaikaran109

TYPES

Control unit generates control signals using one of the two organizations :

- **Hardwired Control Unit.**
- **Micro-programmed Control Unit.**

jaikaran109

Hardwired Control Unit

- Hardwired control units are implemented through use of sequential logic units or circuits like gates , fliflops , decoders in hardware.
- Hardwired control units are generally faster than micro-programmed designs.
- This architecture is preferred in reduced instruction set computers (RISC) as they use a simpler instruction set.
- The hardwired approach has become less popular as computers have evolved as at one time, control units for CPUs were ad-hoc logic, and they were difficult to design.

ADVANTAGES

- Hardwired Control Unit is fast because control signals are generated by combinational circuits.
- The delay in generation of control signals depends upon the number of gates.
- The performances is high as compared to micro-programmed control unit.

DISADVANTAGES

- The control signals required by the CPU will be more complex
- Modifications in control signal are very difficult. That means it requires rearranging of wires in the hardware circuit.
- It is difficult to correct mistake in original design or adding new features in existing design of control unit.

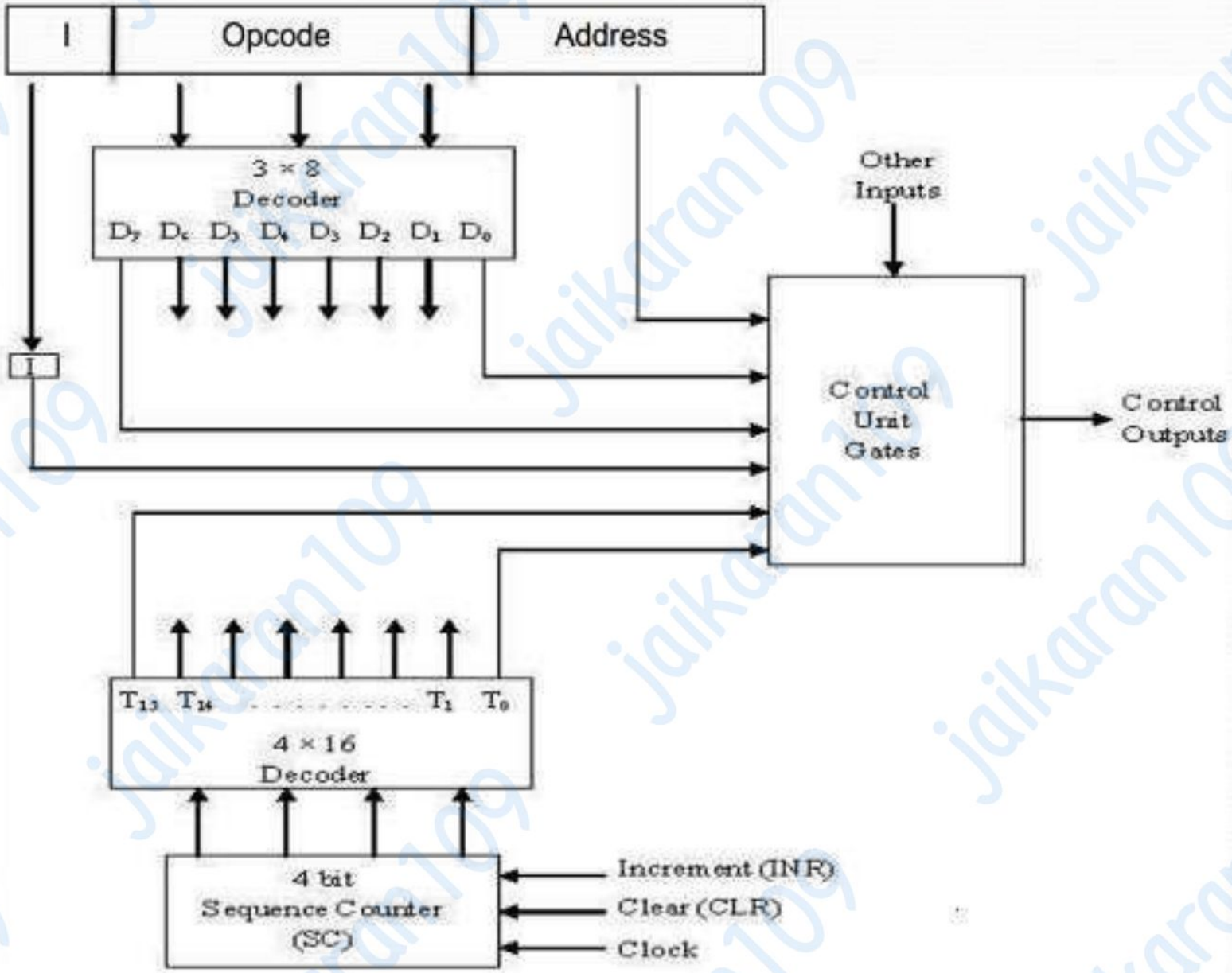
CONSIST OF:

Control unit consist of a:

- Instruction Register
- Number of Control Logic Gates,
- Two Decoders
- 4-bit Sequence Counter

jaikaran109

ARCHITECTURE



- An instruction read from memory is placed in the instruction register (IR).
- The instruction register is divided into three parts: the I bit, operation code, and address part.
- First 12-bits (0-11) to specify an address, next 3-bits specify the operation code (opcode) field of the instruction and last left most bit specify the addressing mode I.
- I = 0 for direct address
- I = 1 for indirect address

- First 12-bits (0-11) are applied to the control logic gates.
- The operation code bits (12 – 14) are decoded with a 3 x 8 decoder.
- The eight outputs (D_0 through D_7) from a decoder goes to the control logic gates to perform specific operation.
- Last bit 15 is transferred to a I flip-flop designated by symbol I.

- The 4-bit sequence counter SC can count in binary from 0 through 15.
- The counter output is decoded into 16 timing pulses T_0 through T_{15}.
- The sequence counter can be incremented by INR.
- input or clear by CLR input synchronously.

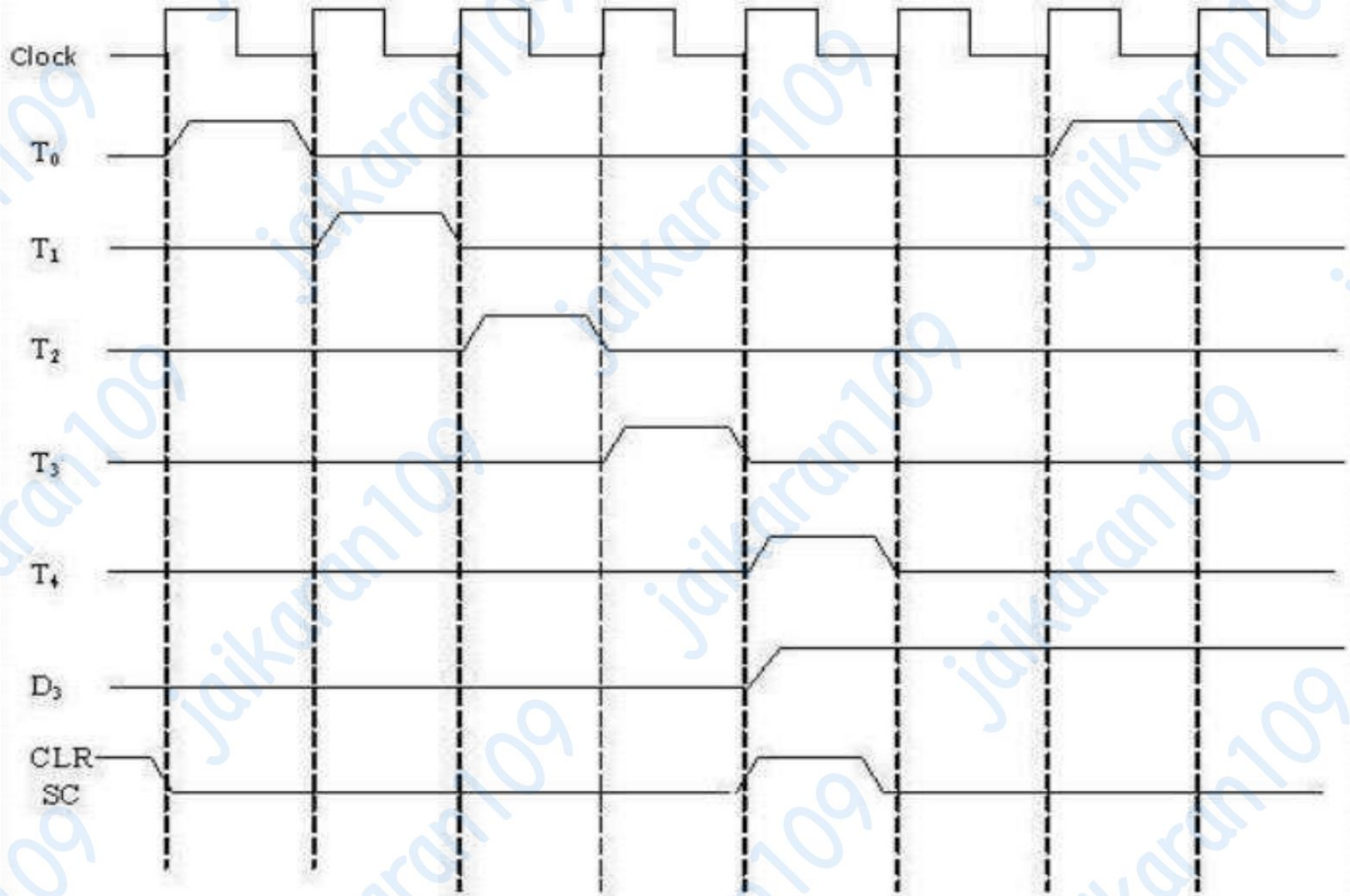
EXAMPLE

- Consider the case where SC is incremented to provide timing signalsT0, T1, T 2 , T3, and T4 in sequence. At time T4 , SC is cleared to 0 if decoder output D3 is active. This is expressed symbolically by the statement:

$$D3\ T4 : SC \leftarrow 0$$

- The timing diagram shows the time relationship of the control signals.

TIMING DIAGRAM



MICRO-PROGRAMMED CONTROL UNIT

- The idea of microprogramming was introduced by Maurice Wilkes in 1951.
- Micro-programs were organized as a sequence of microinstructions and stored in special control memory.
- The main advantage of the micro-program control unit is the simplicity of its structure.
- Outputs of the controller are organized in microinstructions and they can be easily replaced

- A micro-programmed control unit is implemented using programming approach. A sequence of micro operations are carried out by executing a program consisting of micro-instructions.

- Micro-program, consisting of micro-instructions is stored in the control memory of the control unit.

- Execution of a micro-instruction is responsible for generation of a set of control signals.

- **Micro-Programs**: Microprogramming is the concept for generating control signals using programs. These programs are called micro - programs.

- **Micro-Instructions**: The instructions that make micro-program are called micro-instructions.

- **Micro-Code**: Micro-program is a group of microinstructions. The micro- program can also be termed as micro-code.

- **Control Memory**: Micro-programs are stored in the read only memory (ROM). That memory is called control memory.

ADVANTAGES

- The design of micro-program control unit is less complex because micro-
  programs are implemented using software routines.

- The micro-programmed control unit is more flexible because design modifications, correction and enhancement is easily possible.

- The new or modified instruction set of CPU can be easily implemented by
  simply rewriting or modifying the contents of control memory.

- The fault can be easily diagnosed in the micro-program control unit using diagnostics tools by maintaining the contents of flags, registers and counters.

DISADVANTAGES

- The micro-program control unit is slower than hardwired control unit.
- That means to execute an instruction in micro-program control unit requires more time.
- The micro-program control unit is expensive than hardwired control unit
  in case of limited hardware resources.
- The design duration of micro-program control unit is more than hardwired control unit for smaller CPU.

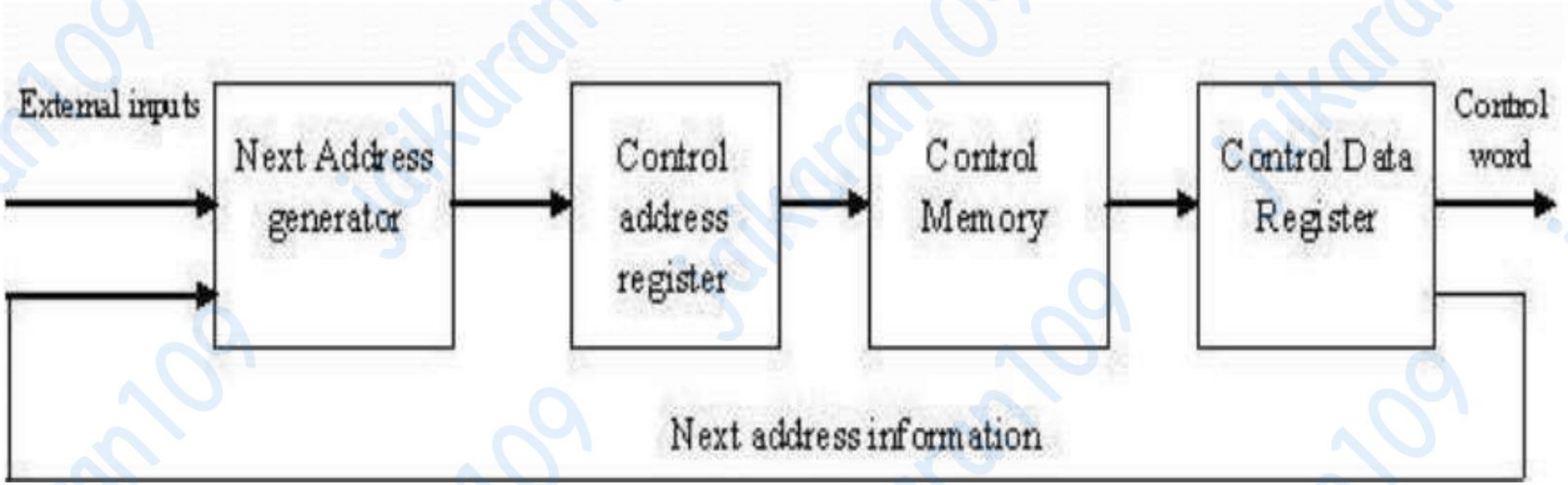
CONSIST OF :

Control unit consist of a :

- Nextaddress generator
- Control address register
- Control memory
- Control data register

ARCHITECTURE

- **The address of micro-instruction that is to be executed is stored in the control address register (CAR).**
- **Micro-instruction corresponding to the address stored in CAR is fetched from control memory and is stored in the control data register (CDR).**
- **This micro-instruction contains control word to execute one or more micro-operations.**
- **After the execution of all micro-operations of micro-instruction, the address of next micro-instruction is located.**


External inputs
Next Address generator
Control address register
Control Memory
Control Data Register
Control word
Next address information

COMPARISON BETWEEN HARDWIRED AND MICROPRGRAMMED CONTROL UNIT

ATTRIBUTE	HARDWIRED CONTROL UNIT	MICRO-PROGRAMMED CONTROL UNIT
SPEED	FAST	SLOW
COST OF IMPLEMENTATION	MORE	CHEAPER
FLEXIBILITY	NOT FLEXIBLE, DIFFICULT TO MODIFY FOR NEW INSTRUCTION	FLEXIBLE, NEW INSTRUCTIONS CAN BE ADDED
ABILITY TO HANDLE COMPLEX INSTRUCTION	DIFFICULT	EASIER
DECODING	COMPLEX	EASY
APPLICATION	RISC MICROPROCESSOR	CISC MICROPROCESSOR
INSTRUCTION SET SIZE	SMALL	LARGE
CONTROL MEMORY	ABSENT	PRESENT
CHIP AREA REQUIRED	LESS	MORE

Hardwired control	Microprogrammed control
✓ The control logic is implemented with gates, flip-flops, decoders, and other digital circuits.	✓ The control information is stored in a control memory. The control memory is programmed to initiate the required sequence of microoperations.
✓ The advantage that it can be optimized to produce a fast mode of operation.	✓ Compared with the hardwired control operation is slow.
✓ Requires changes in the wiring among the various components if the design has to be modified or changed.	✓ Required changes or modifications can be done by updating the microprogram in control memory.

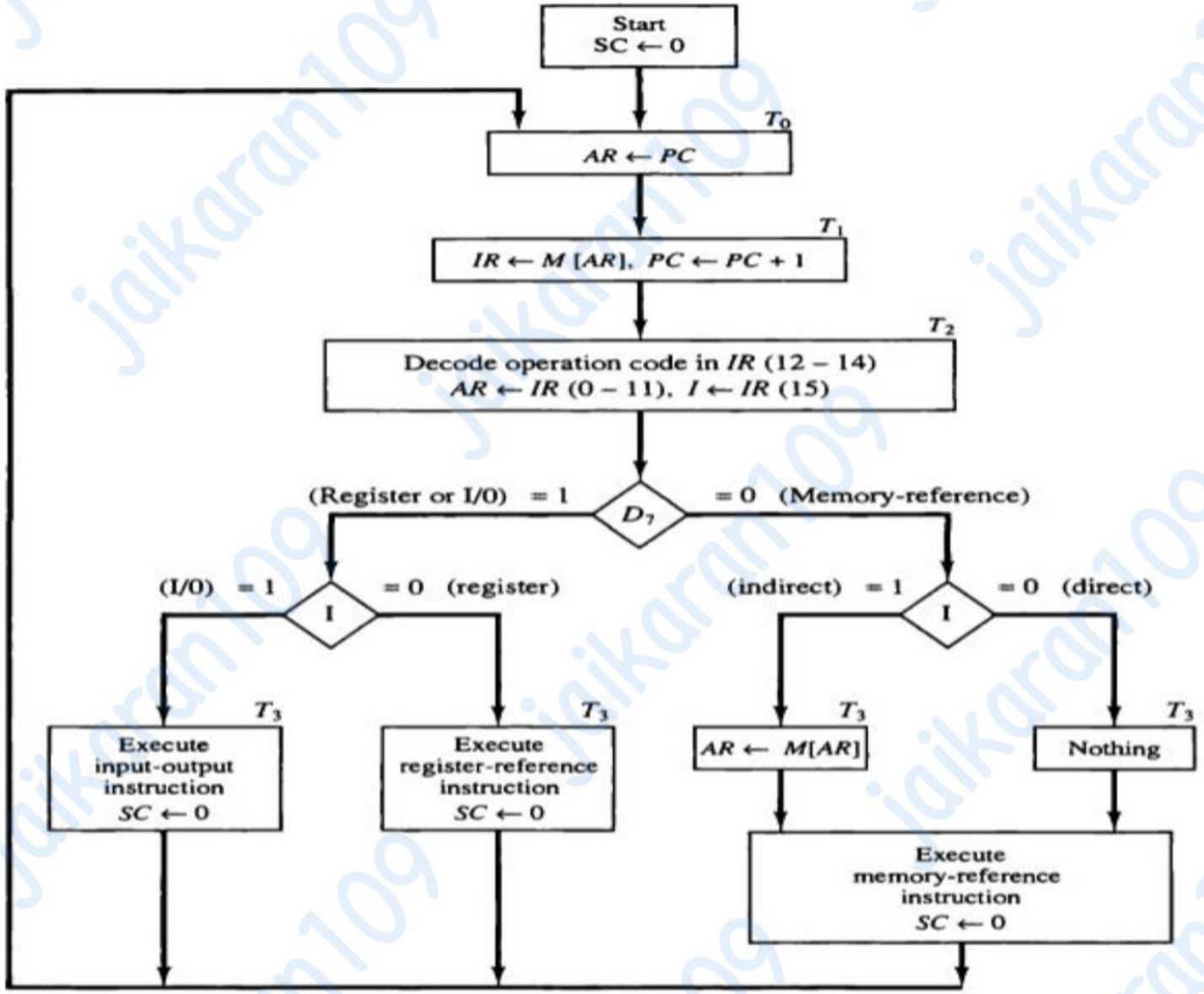
Instruction Cycle

- The program is executed in the computer by going thru a cycle for each instruction.
- In basic computer, each instruction cycle consists of the following phases:

1. Fetch an instruction from memory.
2. Decode the instruction.
3. Read the effective address from memory.
4. Execute the instruction.

jaikaran109

Instruction Cycle



Flowchart for instruction cycle (initial configuration).

Instruction Cycle

- Micro-operations for fetch & decode

T_0: $AR \leftarrow PC$
T_1: $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$
T_2: $D_0, \ldots, D_7 \leftarrow$ Decode $IR(12\text{–}14)$, $AR \leftarrow IR(0\text{–}11)$, $I \leftarrow IR(15)$

jaikaran109

Register-Reference Instructions

- Register-reference instructions are recognized by the control when D7 = 1 and I=0.
- These instructions use bits 0 through 11 of the instruction code to specify one of 12 instructions.
- These 12 bits are available in IR (0-11).

jaikaran109

Memory-Reference Instructions:

TABLE 5-4 Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], \quad E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

- The decoded output Di for i = 0, 1, 2, 3, 4, 5, and 6 from the operation decoder that belongs to each instruction is included in the table.
- The effective address of the instruction is in the address register AR and was placed there during timing signal T2 when I= 0, or during timing signal T3 when I = 1.
- The execution of the memory-reference instructions starts with timing signal T4.

- **AND to AC:**

➢ This is an instruction that performs the AND logic operation on pairs of bits in AC and the memory word specified by the effective address.

➢ The result of the operation is transferred to AC.

➢ The microoperations that execute this instruction are:

$$D_0T_4: \quad DR \leftarrow M[AR]$$
$$D_0T_5: \quad AC \leftarrow AC \wedge DR, \quad SC \leftarrow 0$$

- **ADD to AC:**
- This instruction adds the content of the memory word specified by the effective address to the value of AC.
- The sum is transferred into AC and the output carry Cout is transferred to the E (extended accumulator) flip-flop.
- The microoperations needed to execute this instruction are LDA:

$$D_1T_4: \quad DR \leftarrow M[AR]$$
$$D_1T_5: \quad AC \leftarrow AC + DR, \quad E \leftarrow C_{out}, \quad SC \leftarrow 0$$

LDA: Load to AC

- This instruction transfers the memory word specified by the effective address to AC.
- The microoperations needed to execute this instruction are

$$D_2T_4: \quad DR \leftarrow M[AR]$$

$$D_2T_5: \quad AC \leftarrow DR, \quad SC \leftarrow 0$$

STA: Store AC

- This instruction stores the content of AC into the memory word specified by the effective address.
- Since the output of AC is applied to the bus and the data input of memory is connected to the bus, we can execute this instruction with one microoperation.

$$D_3T_4: \quad M[AR] \leftarrow AC, \quad SC \leftarrow 0$$

- **BUN: Branch Unconditionally**
- This instruction transfers the program to the instruction specified by the effective address.
- The BUN instruction allows the programmer to specify an instruction out of sequence and we say that the program branches (or jumps) unconditionally.
- The instruction is executed with one microoperation:

$$D_4T_4: \quad PC \leftarrow AR, \quad SC \leftarrow 0$$

- **BSA: Branch and Save Return Address**
- This instruction is useful for branching to a portion of the program called a subroutine or procedure.
- When executed, the BSA instruction stores the address of the next instruction in sequence (which is available in PC) into a memory location specified by the effective address.
- The effective address plus one is then transferred to PC to serve as the address of the first instruction in the subroutine.
- This operation was specified with the following register transfer:

$$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$$

Register-Reference Instructions:

- Register-reference instructions are recognized by the control when D7 = 1 and I=0.
- These instructions use bits 0 through 11 of the instruction code to specify one of 12 instructions.
- These 12 bits are available in IR (0-11).
- The control functions and microoperations for the register-reference instructions are listed in Table 5-3.
- These instructions are executed with the clock transition associated with timing variable T3.
- Control function needs the Boolean relation D7I'T3, which we designate for convenience by the symbol r.
- By assigning the symbol Bi to bit i of IR, all control functions can be simply denoted by rBi.

TABLE 5-3 Execution of Register-Reference Instructions

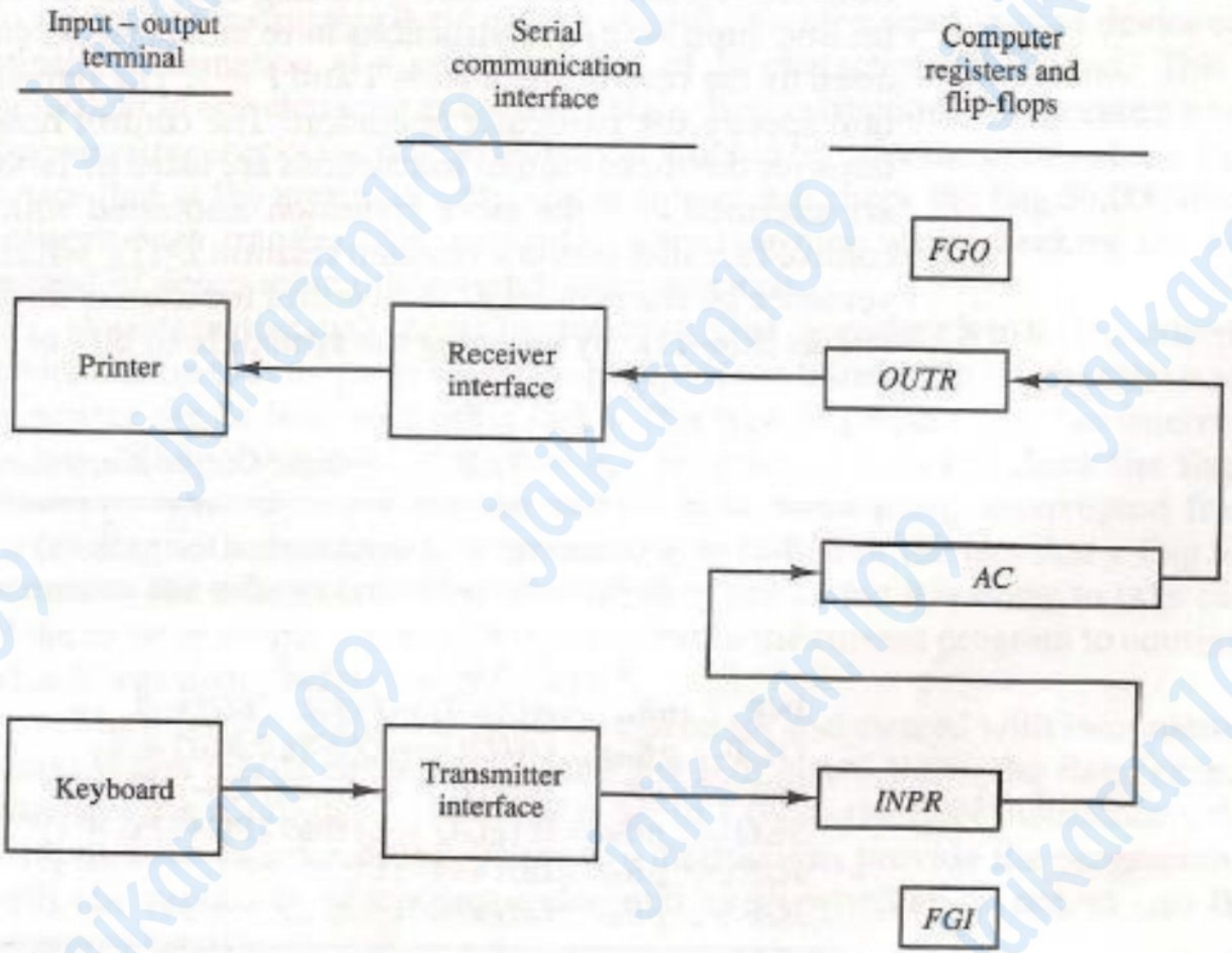
$D_7I'T_3 = r$ (common to all register-reference instructions)
$IR(i) = B_i$ [bit in $IR(0\text{–}11)$ that specifies the operation]

	r:	$SC \leftarrow 0$	Clear SC
CLA	rB_{11}:	$AC \leftarrow 0$	Clear AC
CLE	rB_{10}:	$E \leftarrow 0$	Clear E
CMA	rB_9:	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8:	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7:	$AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	rB_6:	$AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	rB_5:	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1)$	Skip if AC zero
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0:	$S \leftarrow 0$ (S is a start–stop flip-flop)	Halt computer

Input-Output Configuration:

- The terminal sends and receives serial information.
- Each quantity of information has eight bits of an alphanumeric code.
- The serial information from the keyboard is shifted into the input register INPR.
- The serial information for the printer is stored in the output register OUTR.
- These two registers communicate with a communication interface serially and with the AC in parallel.
- The input—output configuration is shown in Fig. 5-12.

Figure 5-12 Input-output configuration.



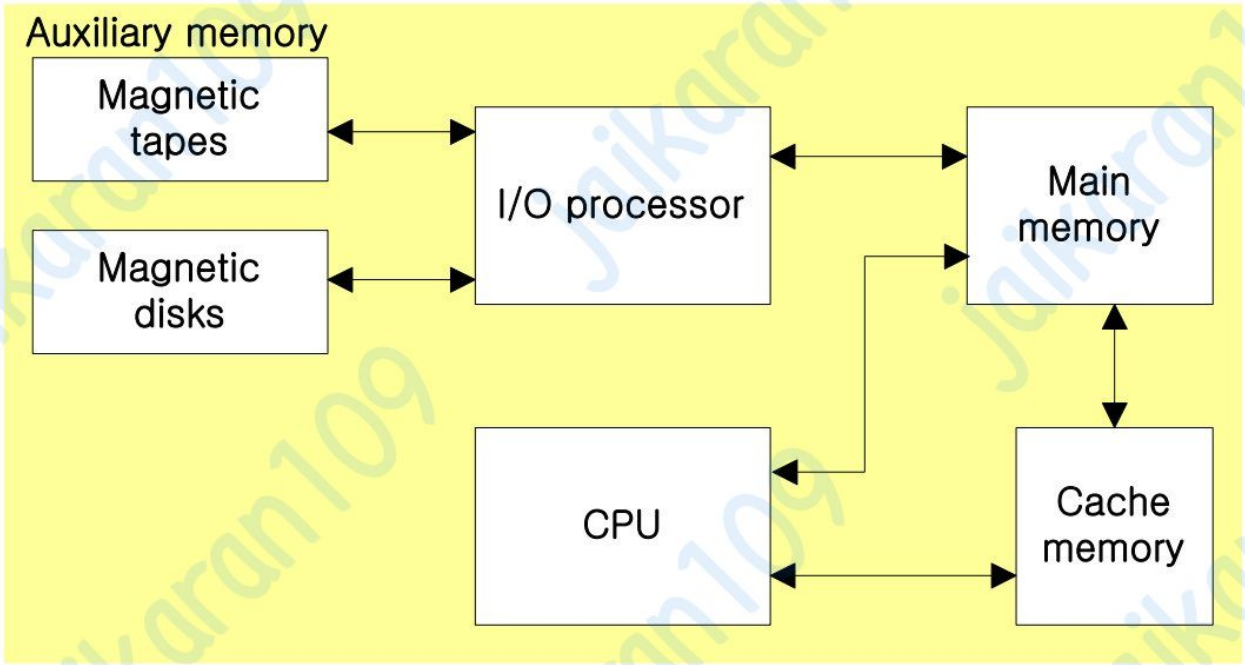
- The input register INPR consists of eight bits and holds alphanumeric input information.
- The 1-bit input flag FGI is a control flip-flop.
- The flag bit is set to 1 when new information is available in the input device and is cleared to 0 when the information is accepted by the computer.
- The output register OUTR works similarly but the direction of information flow is reversed.
- Initially, the output flag FGO is set to 1.
- The computer checks the flag bit; if it is 1, the information from AC is transferred in parallel to OUTR and FGO is cleared to 0.
- The output device accepts the coded information, prints the corresponding character, and when the operation is completed, it sets FGO to 1.

Fundamental Concepts

- Processor fetches one instruction at a time and perform the operation specified.
- Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.
- Processor keeps track of the address of the memory location containing the next instruction to be fetched using Program Counter (PC).
- Instruction Register (IR)

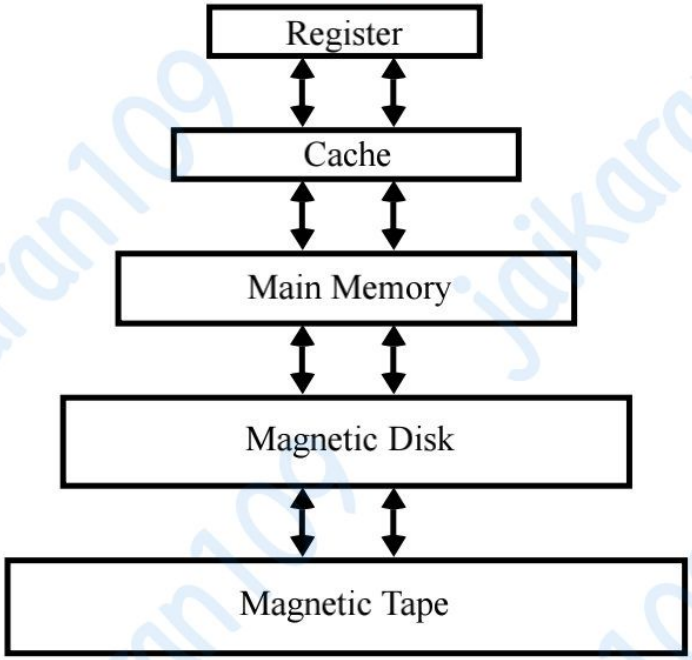
Memory Organization

- Memory Hierarchy
  - Memory hierarchy in a computer system :
    - **Main Memory** : memory unit that communicates directly with the CPU (RAM)
    - **Auxiliary Memory** : device that provide backup storage (Disk Drives)
    - **Cache Memory** : special very-high-speed memory to increase the processing speed (Cache RAM)



Memory Organization

- Multiprogramming
  - enable the CPU to process a number of independent program concurrently
- Memory Management System :
  - supervise the flow of information between auxiliary memory and main memory



- Memory Hierarchy is to obtain the highest possible access speed while minimizing the total cost of the memory system

Random Access Memory (RAM)

- RAM Chips

- Static RAM

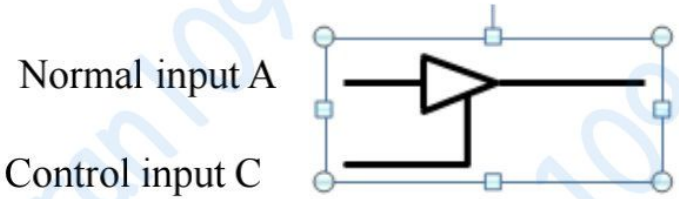
- Consists of Flip-flops to store binary information.
- Static RAM is easier to use and having short Read/Write cycles
- Used mostly in Cache memory.

- Dynamic RAM

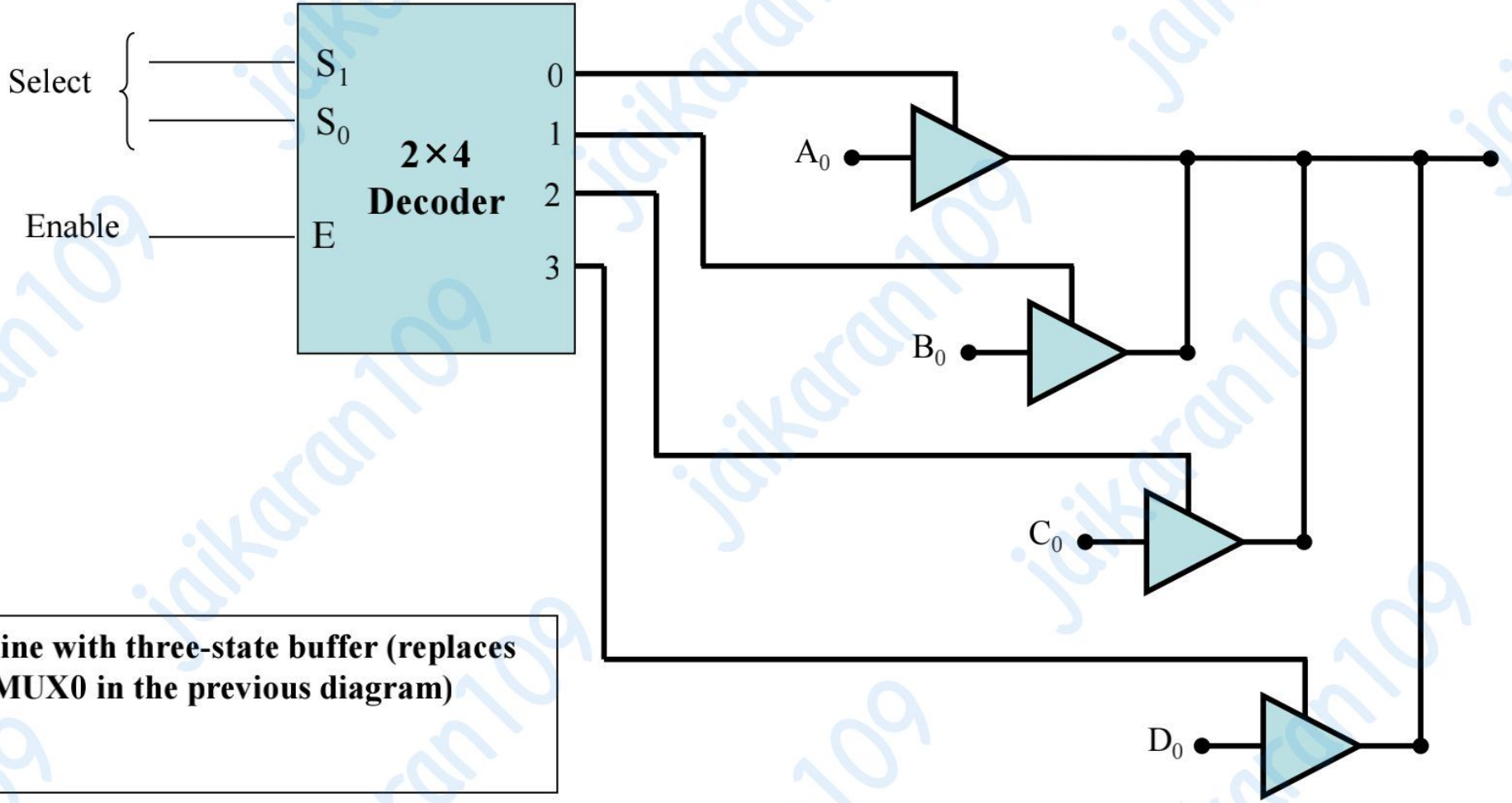
- Stores binary information in the form of electric charge stored inside the capacitor.
- The capacitors are provided by MOS transistors.
- Refreshing circuit is required to refresh the memory.
- Dynamic RAM offers reduced power consumption and large storage capacity.
- It is used to construct main memory.

jaikaran109

Three State Buffer



Output Y=A if C=1
High-impedence if C=0

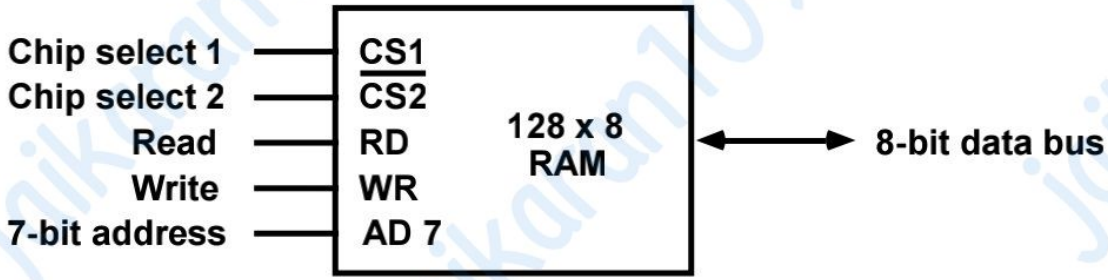


Bus line with three-state buffer (replaces MUX0 in the previous diagram)

MAIN MEMORY

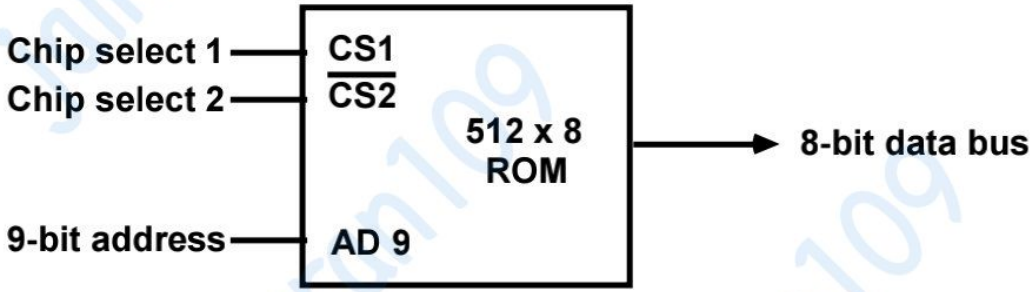
RAM and ROM Chips

Typical RAM chip



CS1	$\overline{CS2}$	RD	WR	Memory function	State of data bus
0	0	x	x	Inhibit	High-impedence
0	1	x	x	Inhibit	High-impedence
1	0	0	0	Inhibit	High-impedence
1	0	0	1	Write	Input data to RAM
1	0	1	x	Read	Output data from RAM
1	1	x	x	Inhibit	High-impedence

Typical ROM chip



MEMORY ADDRESS MAP

Address space assignment to each memory chip

Example: 512 bytes RAM and 512 bytes ROM

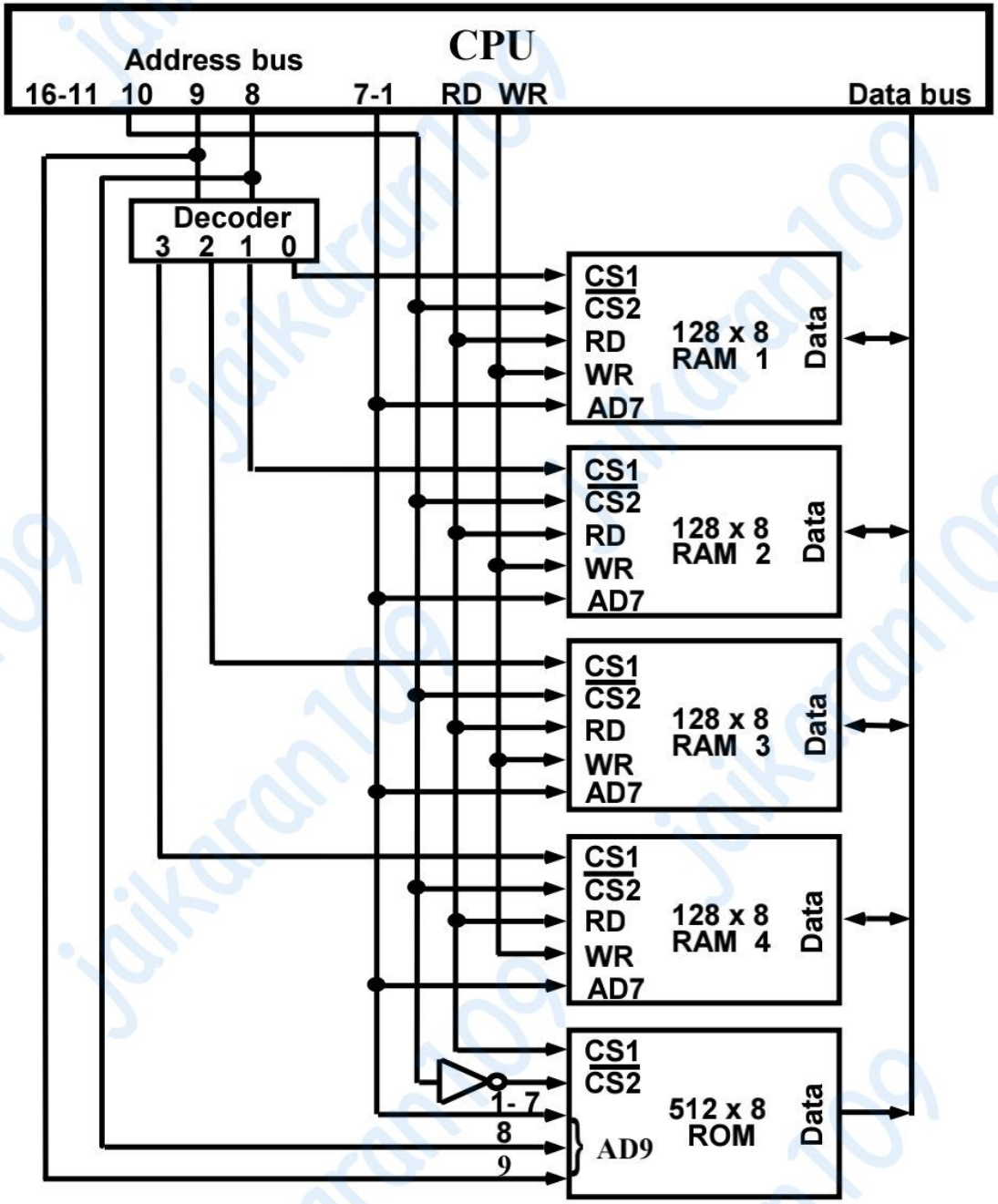
Component	Hexadecimal address	Address bus									
		10	9	8	7	6	5	4	3	2	1
RAM 1	0000 - 007F	0	0	0	x	x	x	x	x	x	x
RAM 2	0080 - 00FF	0	0	1	x	x	x	x	x	x	x
RAM 3	0100 - 017F	0	1	0	x	x	x	x	x	x	x
RAM 4	0180 - 01FF	0	1	1	x	x	x	x	x	x	x
ROM	0200 - 03FF	1	x	x	x	x	x	x	x	x	x

Memory Connection to CPU

- RAM and ROM chips are connected to a CPU through the data and address buses

- The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs

CONNECTION OF MEMORY TO CPU



Numerical Problems

Q1. a) How many 128x8 RAM chips are needed to provide a memory capacity of 2048 bytes.

b) How any lines of the address bus must be used to access 2048 bytes of memory. How many of these lines will be common to all chips.

c) How many lines must be decoded for chip select? Specify the size of decoders.

Q2. Extend the memory system of Fig.1 to 4096 bytes of RAM and 4096 bytes of RAM. List the memory-address map and indicate what size decoders are needed.

Q3. A computer employs RAM chips of 256x8 and ROM chips of 1024x8. The computer system needs 2K bytes of RAM, 4K bytes of ROM, and four interface units each with four registers. A memory-mapped I/O configuration used. The two highest-order bits of the address bus are assigned 00 for RAM, 01 for ROM, and 10 for interface registers.

How many RAM and ROM chips are needed.

Draw a memory-address map for the system.

Give the address range in hexadecimal for RAM, ROM and interface.

CHAPTER 12

12-1 (a) $\frac{2048}{128} = 16$ chips

(b) $2048 = 2^{11}$ — 11 lines to address 2048 bytes
$128 = 2^{7}$ — 7 lines to address each chip
4 lines to decoder for selecting 16 chips

(c) 4×16 decoder

12-4

4096/128 = 32 RAM chips ; 4096/512 = 8 ROM chips.
$4096 = 2^{12}$ — There 12 common address lines + 1 line to select between RAM and ROM.

Component	Address	16 15 14 13	12 11 10 9	8 7 6 5	4 3 2 1
RAM	0000-0FFF	0 0 0 0	← 5×32 decoder →	x x x	x x x x
ROM	1000-1FFF	0 0 0 1	↔ 3×8 decoder	x	x x x x x x x x

to $\overline{CS2}$

Cache Memory

- 12-5 Cache Memory
  - Locality of Reference
    - the references to memory *tend to be confined within a few localized areas* in memory
  - Cache Memory : **a fast small memory**
    - keeping the most frequently accessed instructions and data in the fast cache memory
  - Hit Ratio
    - the ratio of the number of hits divided by the total CPU references (hits + misses) to memory
      - ***hit*** : the CPU finds the word in the cache
      - ***miss*** : the word is not found in cache (CPU must read main memory)
  - A computer with cache access time of 100ns, a main memory access time of 1000 ns, a hit ratio of 0.9 is having average access time of 200ns.

Types of Mapping of Cache Memory

- Mapping
  - The transformation of data from main memory to cache memory
    - 1) Associative mapping
    - 2) Direct mapping
    - 3) Set-associative mapping

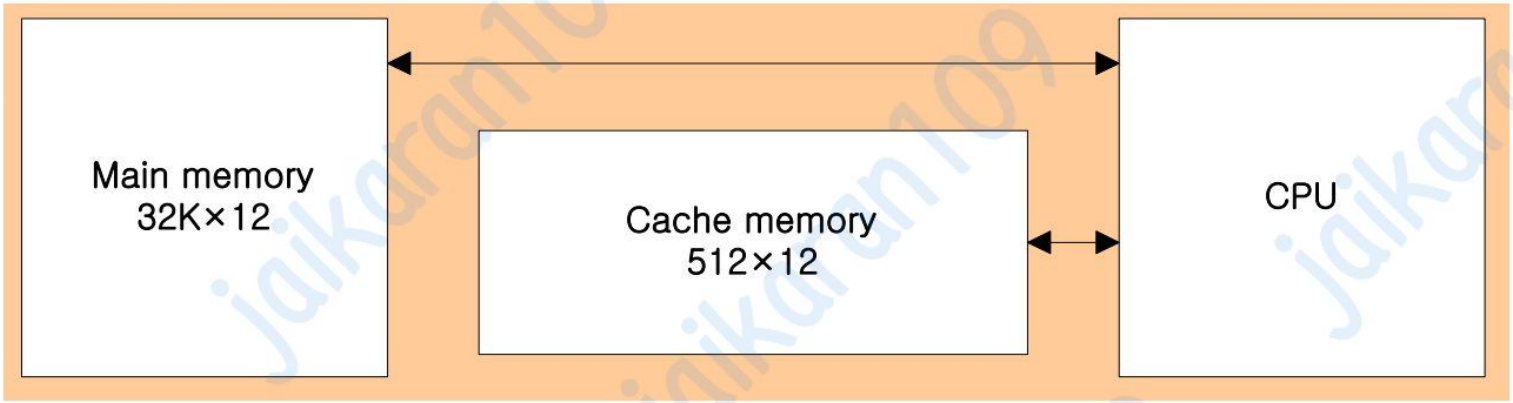
- Associative Mapping:

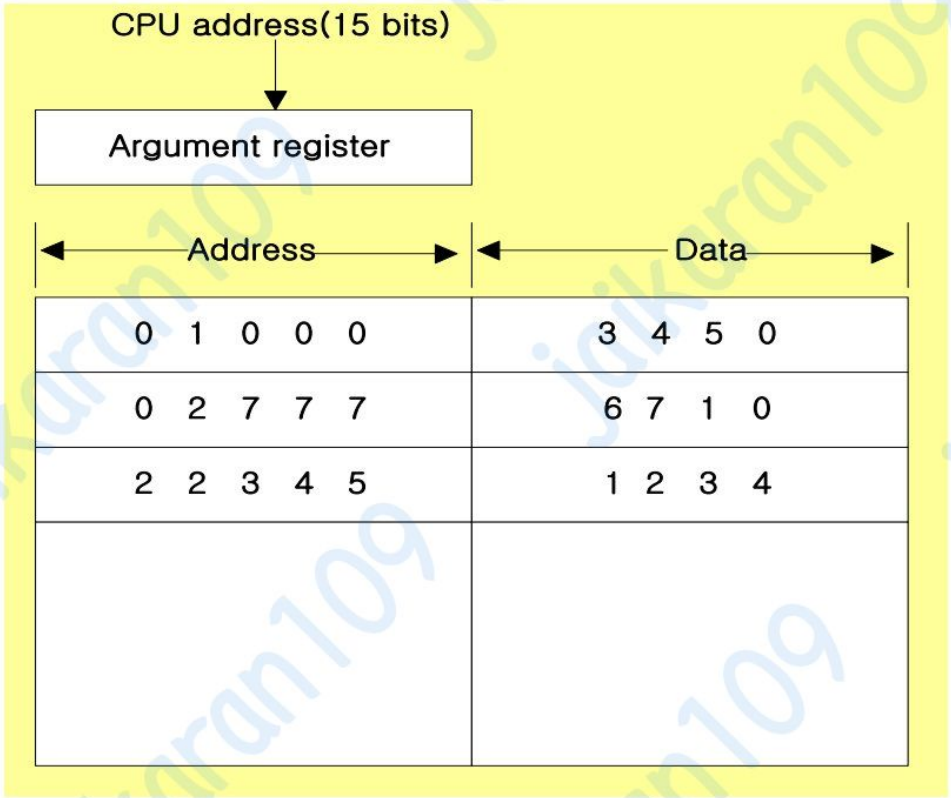
  main memory : **32 K** x 12 bit word (15 bit address lines)

  cache memory : **512** x 12 bit word

    - CPU sends a 15-bit address to cache
      - » **Hit** : CPU accepts the 12-bit data from cache
      - » **Miss** : CPU reads the data from main memory (then data is written to cache)

Associative Mapping





Direct Mapping

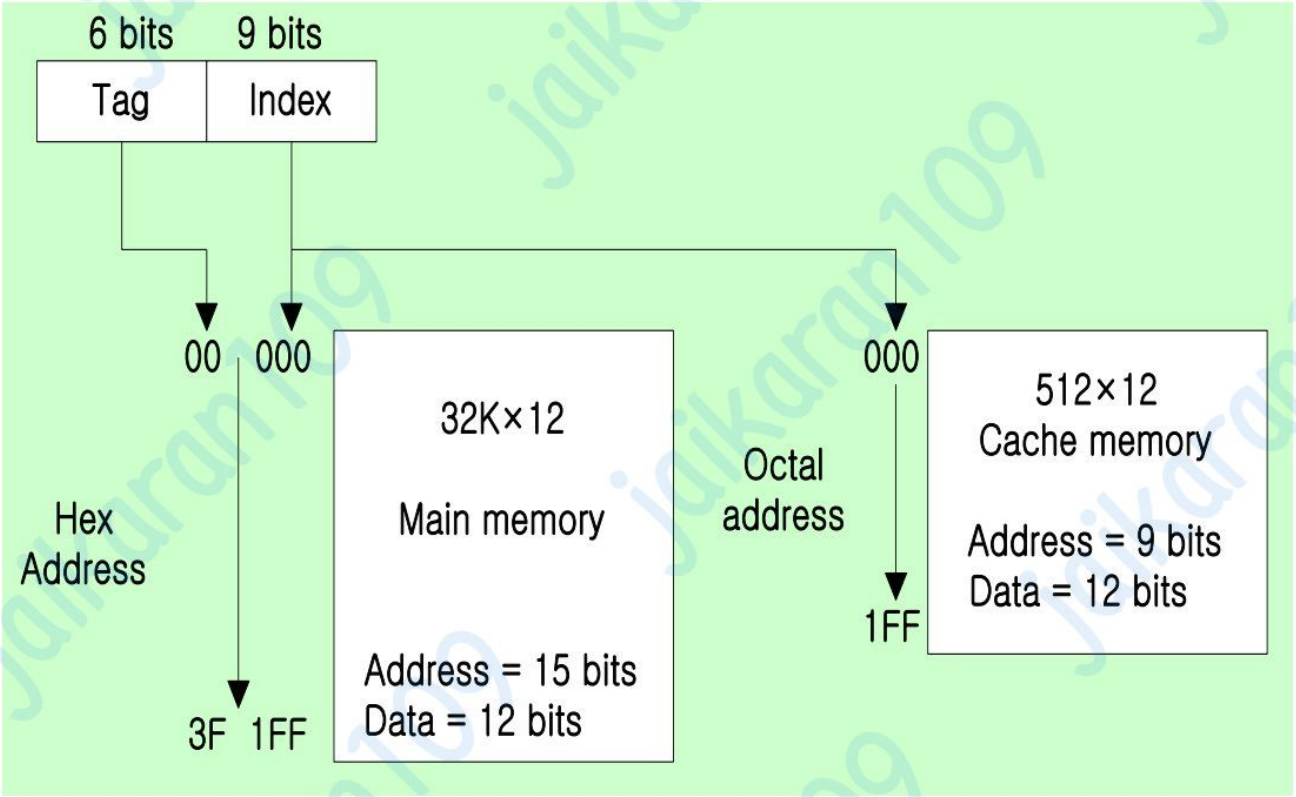
Direct mapping :

n bit memory address

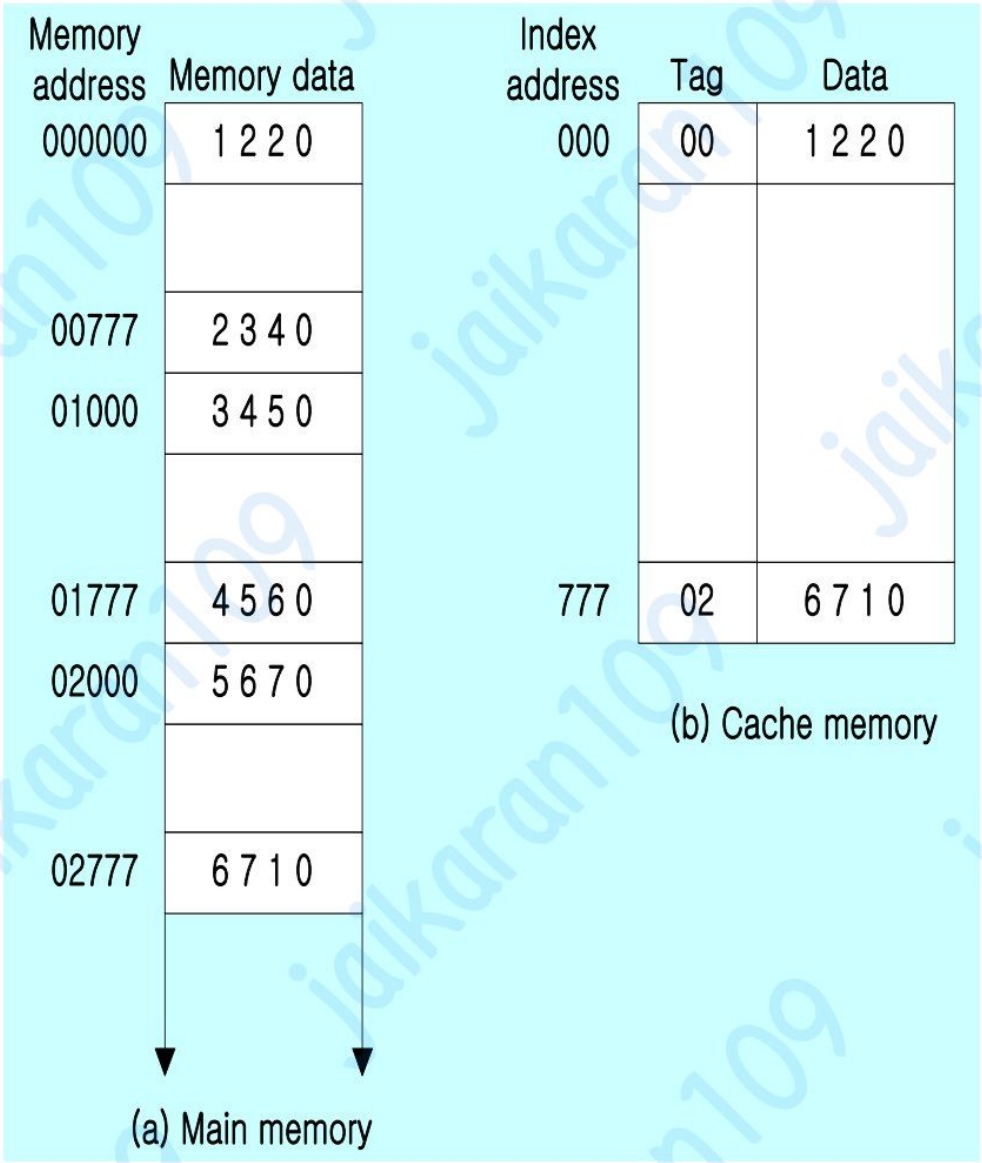
Tag field (**n - k**) : Index field (**k**)

2^k words cache memory + 2^n words main memory

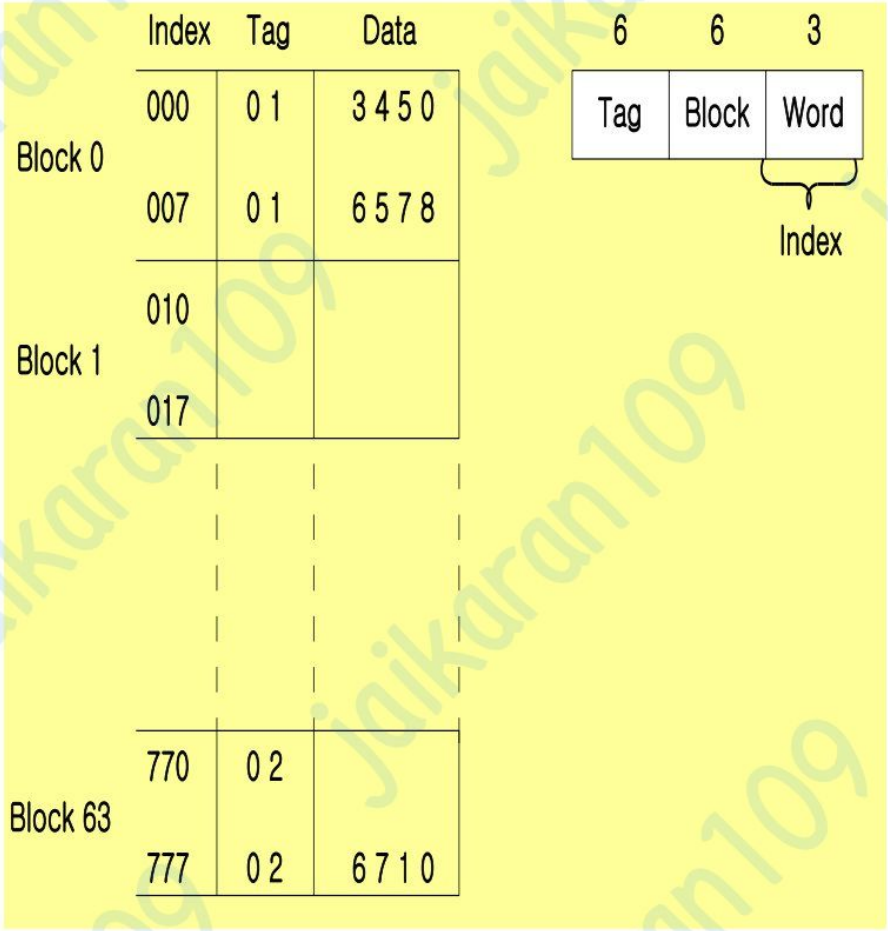
Tag = 6 bit (15 - 9), **Index** = 9 bit



Direct Mapping



- Direct mapping cache with block size of 8 words :
  - » 64 block x 8 word = 512 cache words size



Set-Associative Mapping

Set-associative mapping : (two-way)

Index	Tag	Data	Tag	Data
000	0 1	3 4 5 0	0 2	5 6 7 0
777	0 2	6 7 1 0	0 0	2 3 4 0

Questions

- A two-way set associative cache memory uses blocks of four words. The cache can accommodate a total of 2048 words from main memory. The main memory size is 128Kx32. Formulate all pertinent information required to construct the cache memory. What is the size of cache memory.

- A computer has a memory unit of 64Kx16 and a cache memory of 1K words. The cache uses direct mapping with a block size of four words. How many bits are there in the tag, index, block, and words fields of the address format. How many bits are there in each word of cache, and how are they divided into functions? Include a valid bit. How many blocks can the cache accommodate?

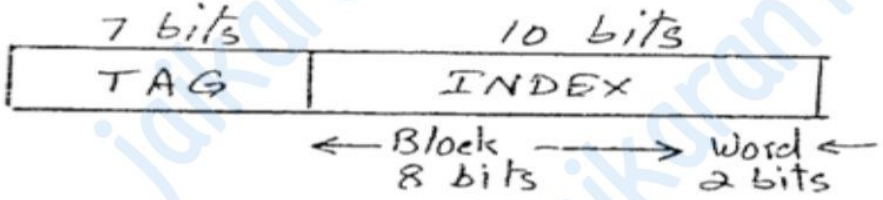
- The access time of a cache memory is 100ns and that of main memory is 1000ns. It is estimated that 80 percent of the memory requests are for read and the remaining 20 percent for write. The hit ratio for read access only is 0.9. A write-through procedure is used. What is the average access time of the system considering only memory read cycles. What is the average access time of the system for both read and write requests. What is the hit ratio considering the write requests also.

Answer

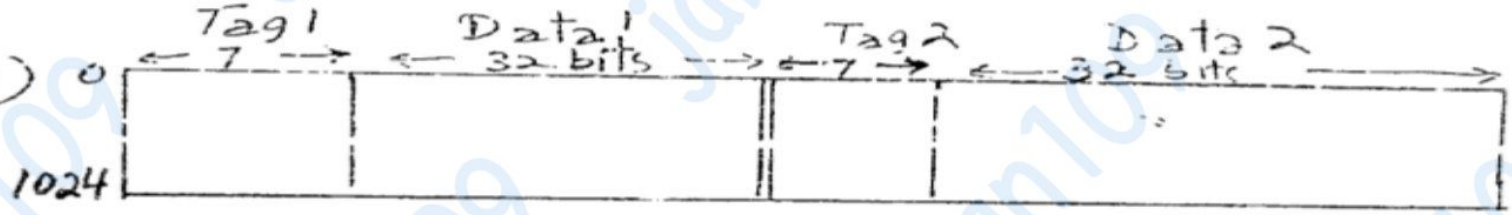
12-15

$128K = 2^{17}$; For a set size of 2, the index address has 10 bits to accomodate $\frac{2048}{2} = 1024$ words of cache.

(a)



(b)



Size of cache memory is $1024 \times 2(7+32)$
$= 1024 \times 78$

12-16

(a) $0.9 \times \underbrace{100}_{\text{cache access}} + 0.1 \times \underbrace{11000}_{\text{cache+memory access}} = 90 + 110 = 200$ nsec.

(b) $0.2 \times \underbrace{1000}_{\text{write access}} + 0.8 \times \underbrace{200}_{\text{read access from (a)}} = 200 + 160 = 360$ nsec.

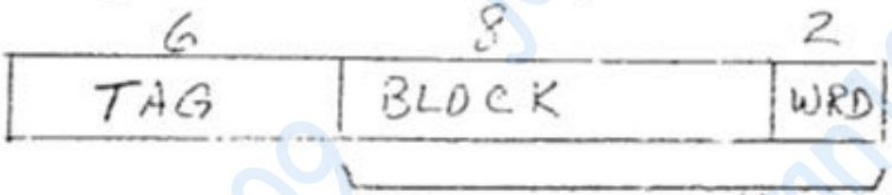
(c) Hit ratio $= 0.8 \times 0.9 = 0.72$

Answer

12-18

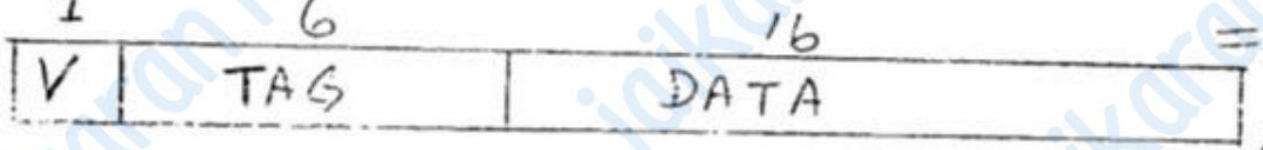
64K × 16 : 16 bit address; 16-bit data.

(a)



= 16 bit address

(b)



= 23 bits in each word of cache

(c) $2^8 = 256$ blocks of 4 words each

- Replacement Algorithm : cache miss or full
  - 1) **LRU** (Least Recently Used):
  - 2) **Random Replacement:**
  - 3) **FIFO** (First-In First-Out) :

- Writing to Cache :
  - 1) **Write-through** :
  - 2) **Write-back** :

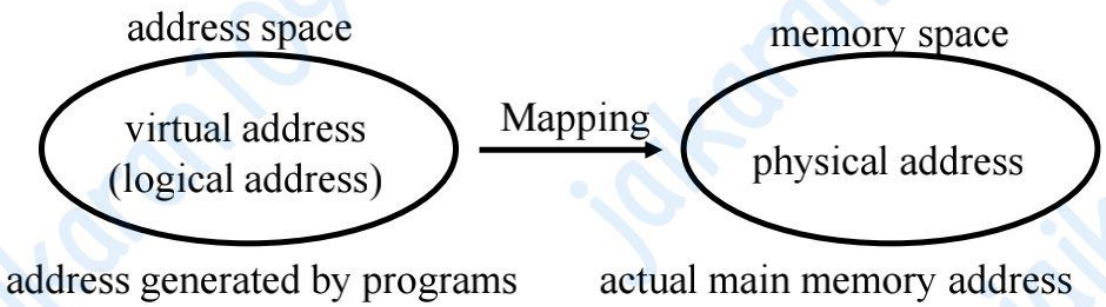
- In **write-through**, data is **simultaneously updated to cache and memory**. This process is simpler and more reliable. This is used when there are no frequent writes to the cache(The number of write operations is less).
- It helps in data recovery (In case of a power outage or system failure). A data write will experience latency (delay) as we have to write to two locations (both Memory and Cache).
- **In write back-**
- The data is updated only in the cache and updated into the memory at a later time. Data is updated in the memory only when the cache line is ready to be replaced.

- Cache Initialization
  - Cache is initialized :
    - 1) When power is applied to the computer
    - 2) When main memory is loaded with a complete set of programs from auxiliary memory
    - 3) Cache is initialized by clearing all the valid bits to 0.
  - Valid bit
    - Indicate whether or not the word contains valid data

VIRTUAL MEMORY

Give the programmer the illusion that the system has a very large memory, even though the computer actually has a relatively small main memory

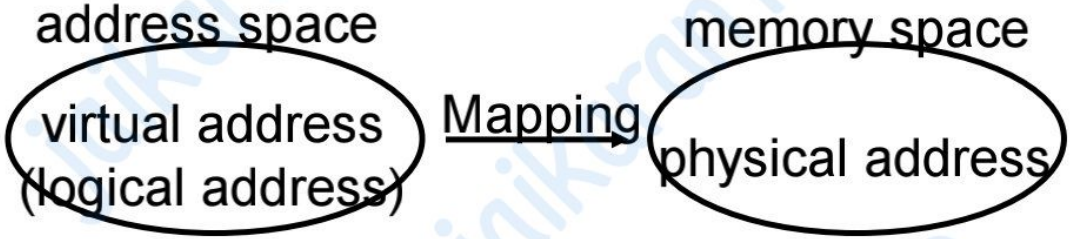
Address Space(Logical) and Memory Space(Physical)



VIRTUAL MEMORY

Give the programmer the illusion that the system has a very large memory, even though the computer actually has a relatively small main memory

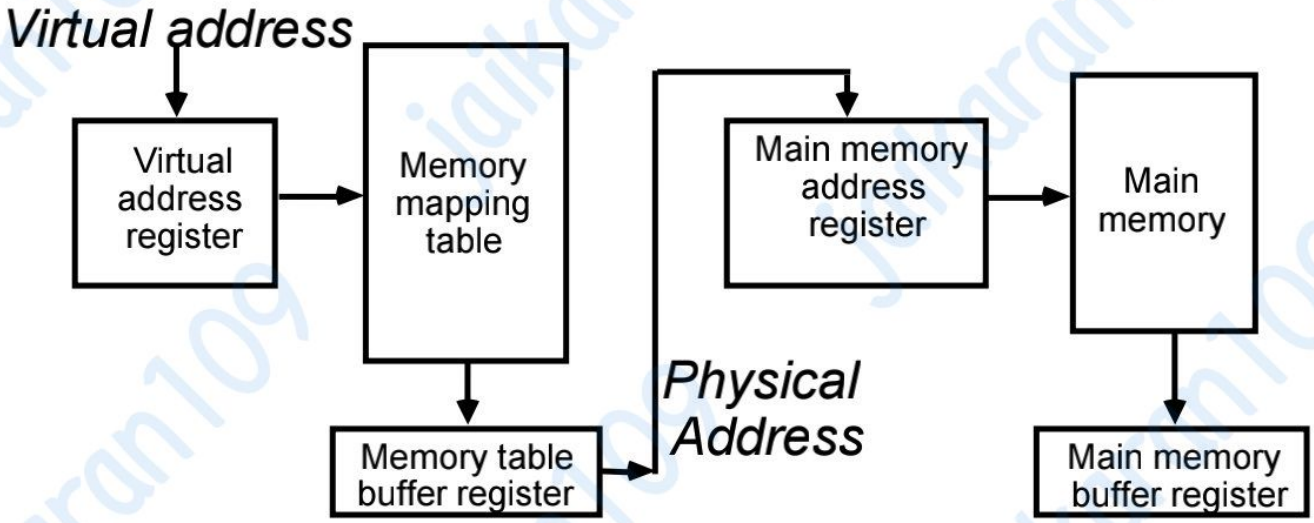
Address Space(Logical) and Memory Space(Physical)



address generated by programs — actual main memory address

Address Mapping

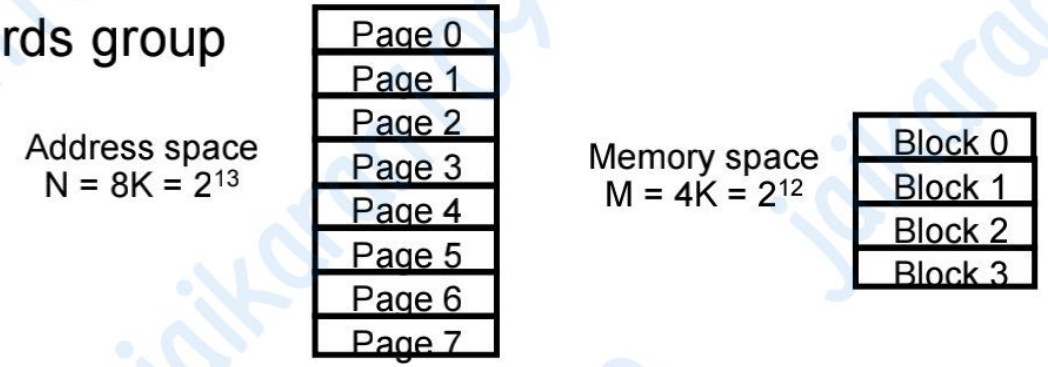
Memory *Mapping Table* for *Virtual Address -> Physical Address*



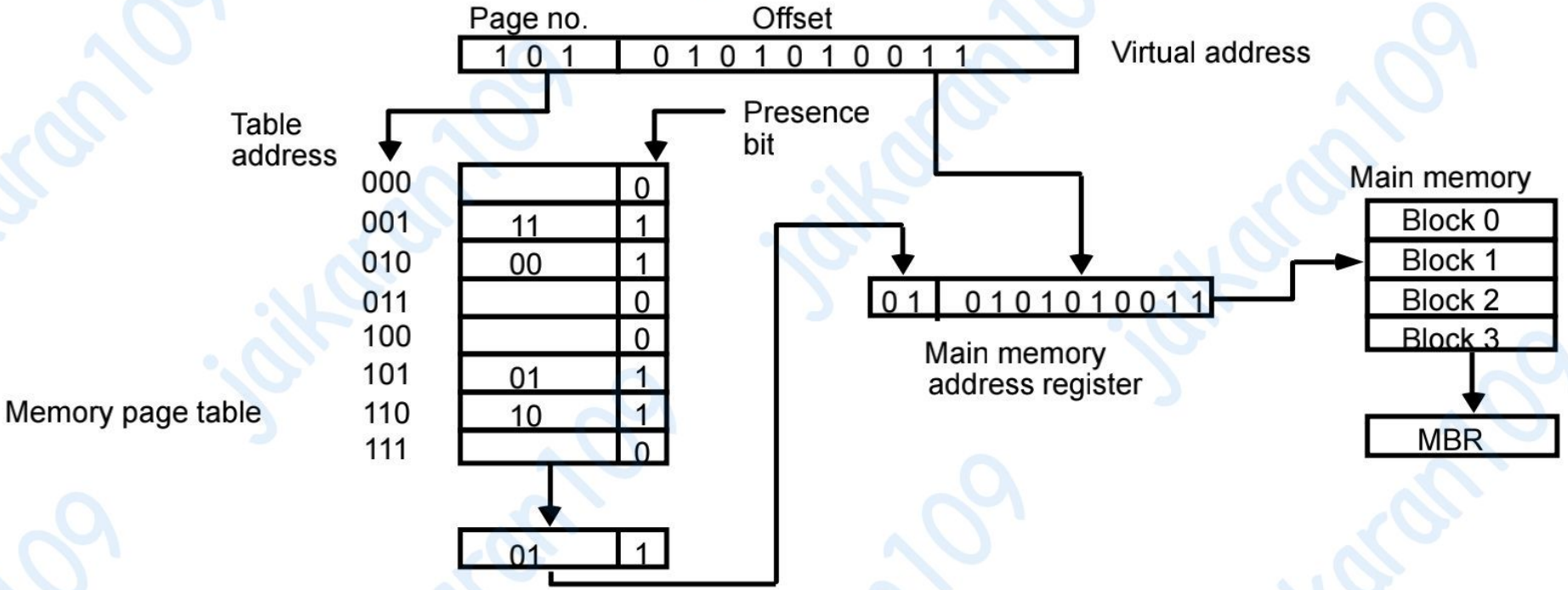
ADDRESS MAPPING

Address Space and Memory Space are each divided into fixed size group of words called *blocks* or *pages*

1K words group

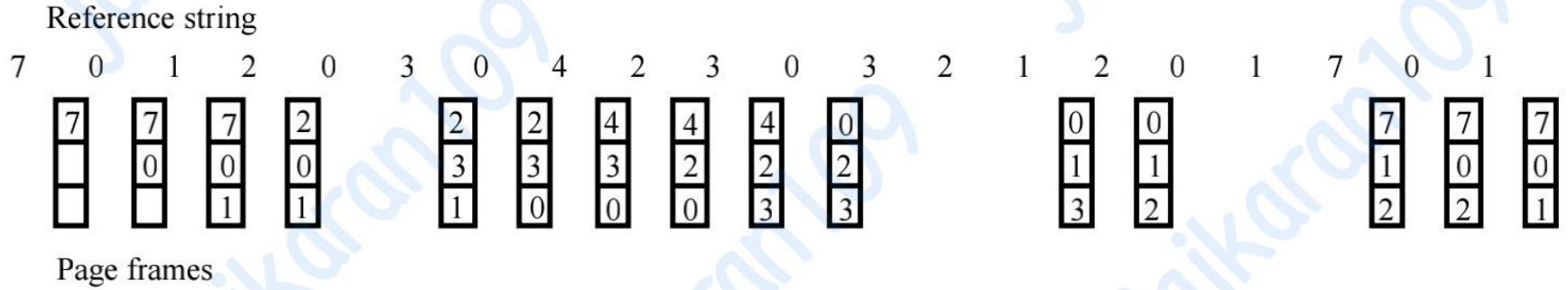


Organization of memory Mapping Table in a paged system



PAGE REPLACEMENT ALGORITHMS

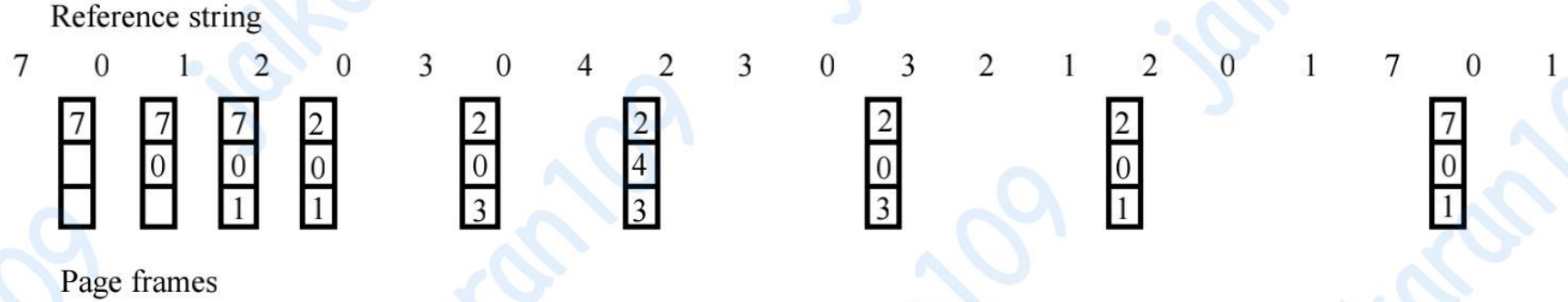
FIFO



FIFO algorithm selects the page that has been in memory the longest time

Optimal Replacement (OPT) - Lowest page fault rate of all algorithms

> Replace that page which will not be used for the longest period of time

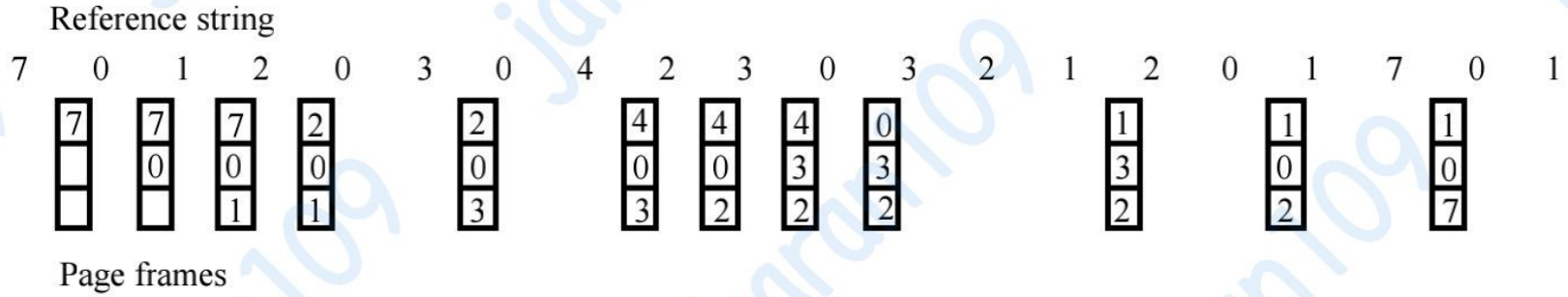


PAGE REPLACEMENT ALGORITHMS

LRU

- OPT is difficult to implement since it requires future knowledge

> Replace that page which has not been used for the longest period of time



- A virtual memory system has an address space of 8K words, a memory space of 4K words and page & block sizes are of 1K words. The following page is referenced in memory: 4 2 0 1 2 6 1 4 0 1 0 2 3 5 7. Determine the four pages that are resident in main memory after each page reference change if page replacement algorithm is : FIFO, LRU, Optimal.

- An address space is specified by 24 bits and the corresponding memory space by 16 bits. How many words are there in the address space. How many words are there in the memory space. If a page consists of 2K words, how many pages and blocks are there in the system?

(a) Address space = 24 bits 2^{24} = 16 M words

(b) Memory space = 16 bits 2^{16} = 64 K words

(c) $\frac{16M}{2K}$ = 8 K pages $\frac{64K}{2K}$ = 32 blocks

Question

The page request are in following order. If there are three memory frames, show the execution of the following pages using

a. FIFO b. Optimal c. LRU

6 0 1 3 2 5 0 4 7 1 3 2 6 0 4

jaikaran109

Input-Output Organization

- **Peripheral Devices**: Input & Output devices attached to computer are called Peripheral.

Input Devices:

- Keyboard, Optical devices: Card reader, Bar code reader etc.
- Magnetic Input Devices: Magnetic stripe reader
- Screen Input Devices: Touch screen, Light pen etc.

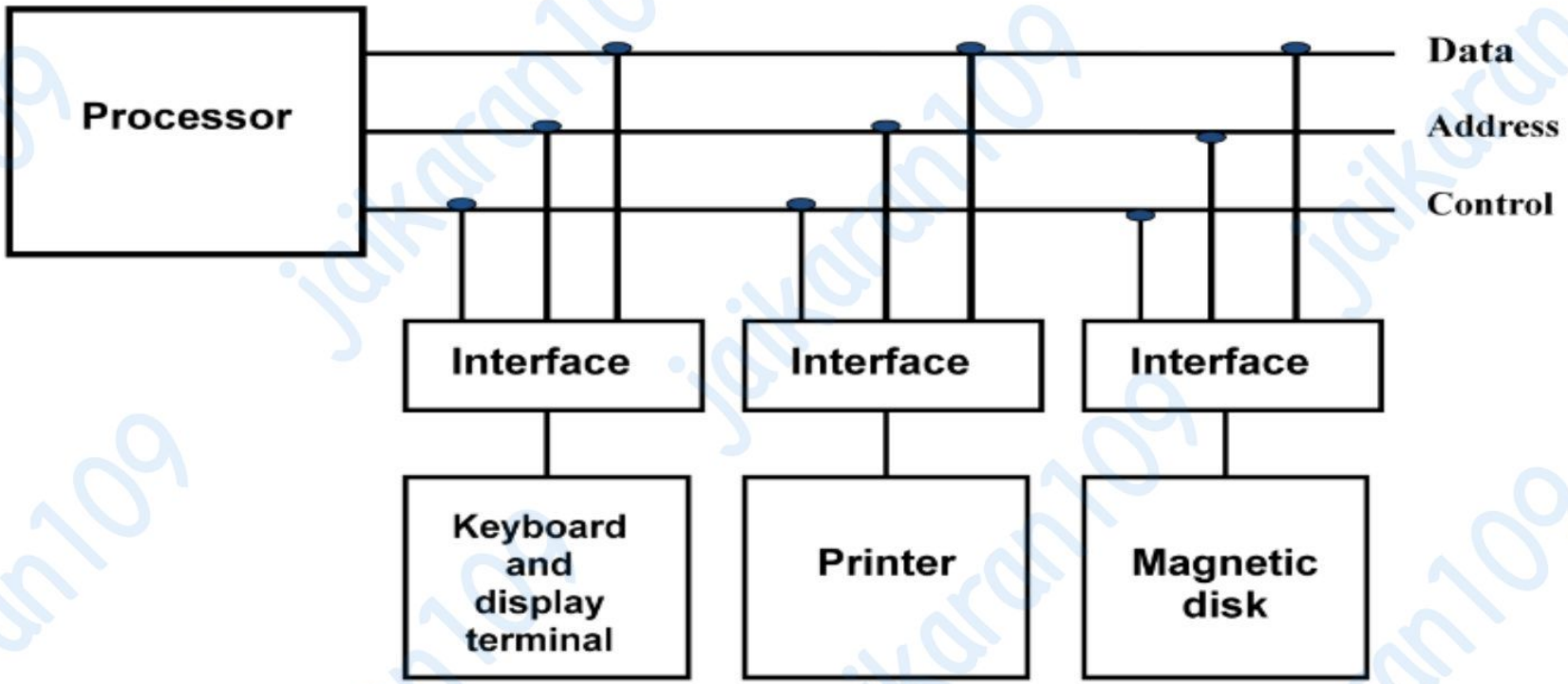
Output Devices:

- Keyboard, Optical devices: Card reader, Bar code reader etc.
- Peripherals that provide auxiliary storage are magnetic tape and magnetic disk.
- Input and output devices communicate alphanumeric information by using ASCII 7bit code.
- To use computer efficiently, large number of programs and data must be prepared in advance for execution with computer.

Input-Output Interface

- I/O interface provides a method for transferring information between internal storage and external I/O devices.
- The purpose of interfacing are as follows:
  - Peripherals are electromechanical & electromagnetic devices and are interacting with electronics devices(CPU).
  - Data transfer rate of peripherals is usually slower than transfer rate of CPU.
  - Data codes and formats in peripherals differ from word format in CPU
  - Operating modes of peripherals are different from each other and each one must be controlled without disturbing other.
- To resolve these differences, computer system includes Interface units between CPU and peripherals to supervise and synchronize all input and output transfers.

Input-Output Interface



Connection of I/O bus to input-output devices

- Each peripheral has its own controller that operates a particular electromechanical device.
- To communicate with a particular device, the processor places a device address on the address lines.

Input-Output Interface

- When the interface detects it own address, it activates path between bus lines and the device.

- At the time address is made available in address lines, the processor provides function code(I/O command) in the control lines.

- Types of I/O command:

➢ Control command: To activate and inform what to do.

➢ Status command: To test various status conditions.

➢ Data Output data: It causes the transfer of data from bus into one of its registers.

➢ Data Input Command: Interface receives data from peripheral and places them on buffer register where it is put into data lines.

I/O versus Memory Bus

- In addition to communicating to I/O, processor must also communicate with memory unit.

- There are three ways that computer buses can be used to communicate with memory and I/O:
  - Use two separate buses, one for memory and one for I/O. (**IOP**)
  - Use one common bus for both memory and I/O but have separate control lines for each. (**Isolated I/O**)
  - Use one common bus for memory and I/O with common control lines. (**Memory Mapped I/O**)

Isolated I/O

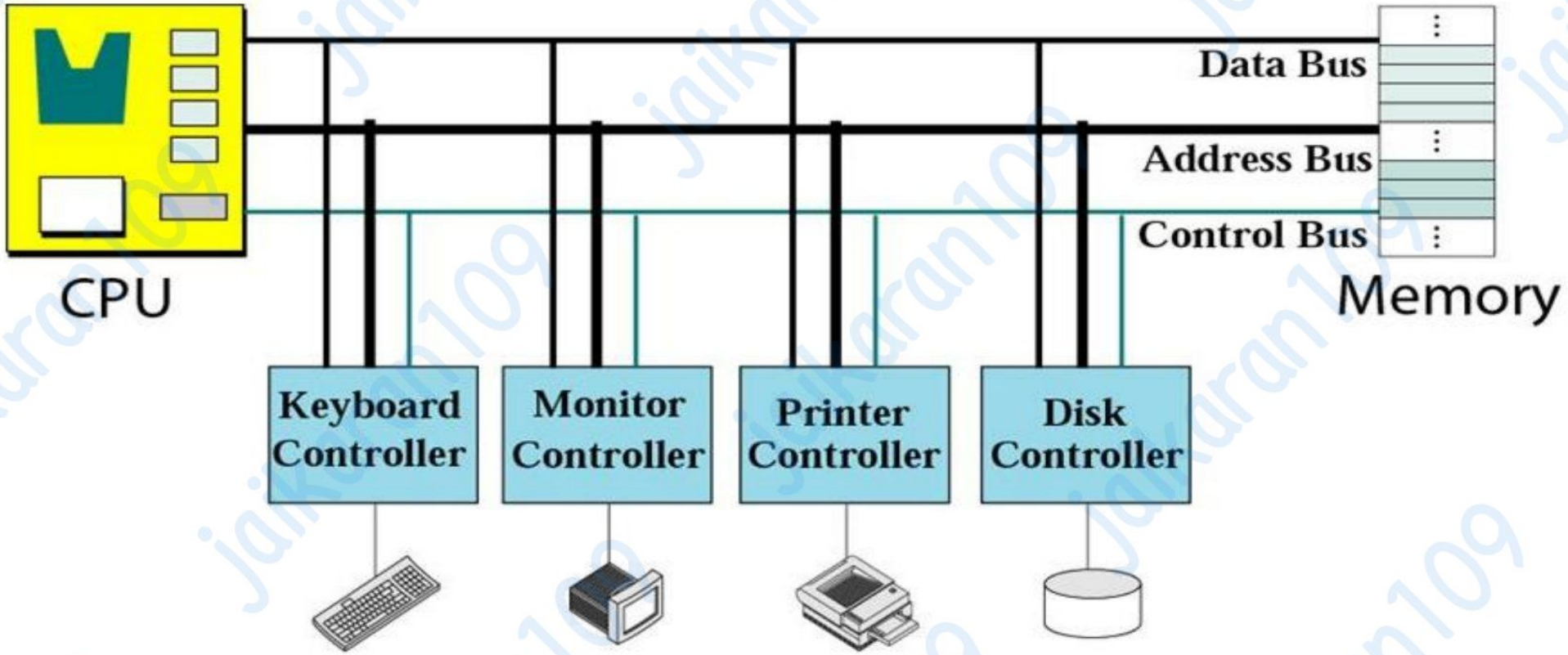
- The distinction between memory and I/O transfer is made through separate read and control lines.
- I/O read and I/O write are enabled during I/O transfer and Memory read/write are enabled during memory transfer.
- In the isolated I/O configuration, CPU have distinct input and output instructions where each of it will be associated with address of the interface register.
- When the CPU fetches and decodes the I/O instruction, it places the address associated with the instruction on the common address lines and enables I/O read or I/O write control line.
- When the CPU fetches and decodes the Memory instruction, it places the address associated with the instruction on the common address lines and enables Memory read or Memory write control line.

Memory Mapped I/O

- Here the same address space is used for both memory and I/O. The computer treats interface register as a part of memory system.
- The assigned address cannot be used for storing memory words, which reduces memory address range available.
- In a memory-mapped I/O organization, there are no specific, input or output instruction.
- CPU manipulate I/O data residing in interface registers with the same instruction used to manipulate memory words.
- Load and store instruction used for reading/writing from memory and can be use for input or output data from I/O instruction.
- With memory-mapped I/O all instructions that refer to memory are also available for I/O.

Memory Mapped I/O

Connecting I/O devices to the buses



Modes of Transfer

- Data transfer between the CPU and the I/O devices may be handled in variety of modes. Some modes use the CPU as an intermediate path and others transfer the data directly to and from the memory unit.

- Data transfer to and from peripherals may be handled in three ways:
  - Programmed I/O
  - Interrupt-initiated I/O
  - Direct Memory Access (DMA)

jaikaran109

Programmed I/O

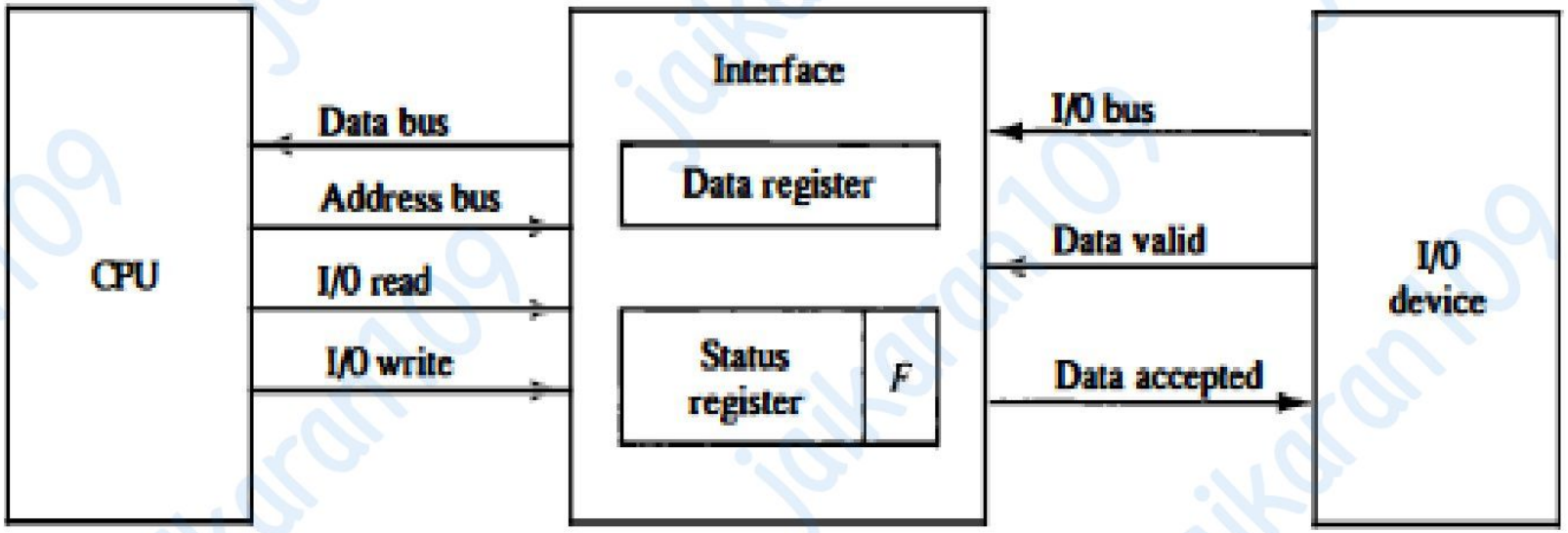
- Programmed I/O operations are the result of I/O instructions. Each data item transfer is initiate by an instruction in the program.

- Usually the transfer is to and from a CPU register and peripheral. Other instructions are needed to transfer data between memory and CPU.

- Once a data transfer is initiated, the CPU is required to monitor the interface to see when a transfer can again be made.

Programmed I/O

Figure 11-10 Data transfer from I/O device to CPU.



F = Flag bit

Interrupt Initiated I/O

- In the programmed I/O CPU stays in program loop until the I/O indicates that it is ready for data transfer.
- This is time consuming process since it makes CPU busy needlessly.
- This can be avoided by using an interrupt facility.
- When the interface determines that device is ready for data transfer, it generates an interrupt request.

- Upon detecting external interrupt signal, the CPU momentarily stops the task it is processing.

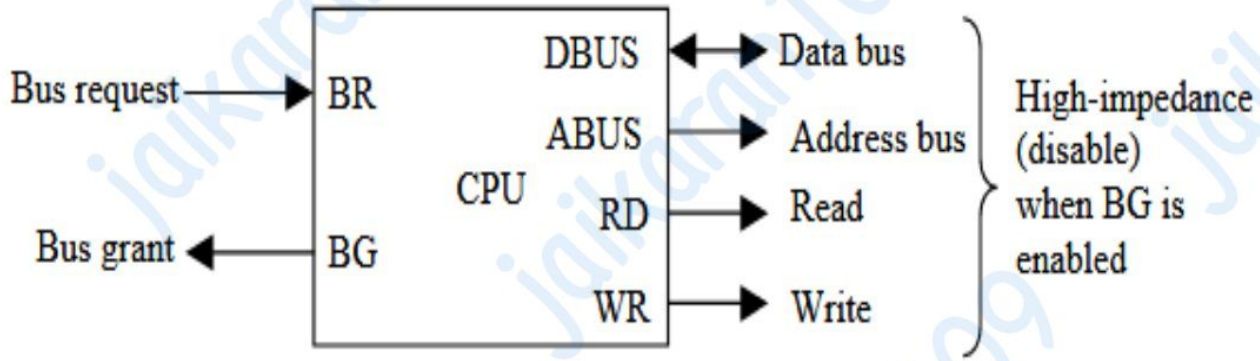
- It then branches to fulfill the I/O request and return to the original task.
- Based on the concept of Priority Interrupt.

Direct Memory Access (DMA)

- The transfer of data between a fast storage device such as magnetic disk and memory often limited to the speed of CPU.
- Removing the CPU and letting the peripheral device manage the memory bus directly improve speed of transfer.
- Such transfer technique is called Direct Memory Access (DMA).
- A DMA controller takes over the buses to manage the transfer directly between I/O device and memory.
- During DMA transfer, the CPU is idle and has no control over memory buses.
- By using **Bus Request(BR)** and **Bus Grant(BG)** the buses are released to DMA controller.

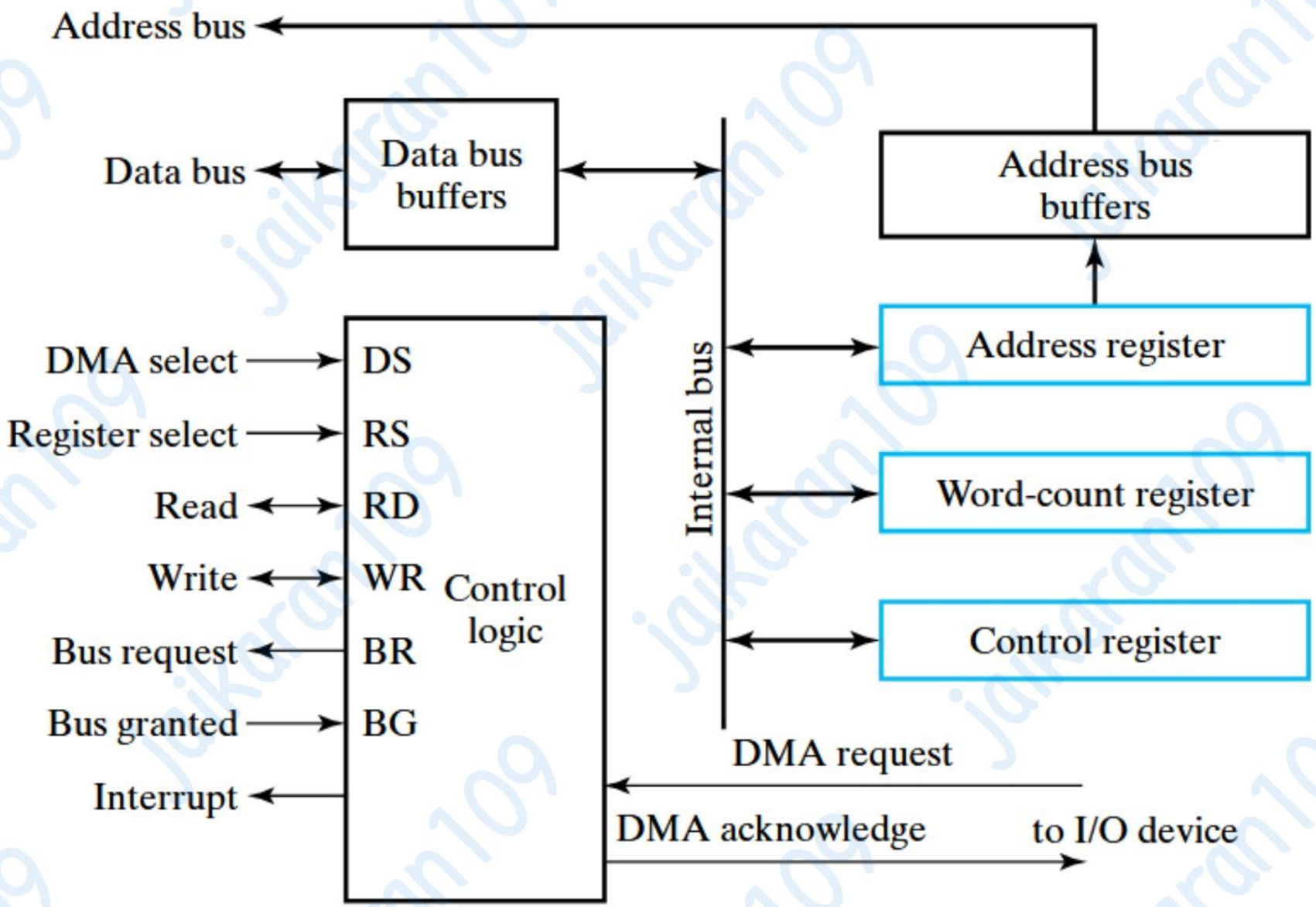
DMA

Figure 6.13 CPU bus signals for DMA transfer.



- Data transfer ways:
  - Burst Transfer: Here number of words are transferred in a block. Example: Magnetic disk.
  - Cycle stealing: Allows the DMA controller to transfer one data word at a time after it must return the control of buses to CPU.

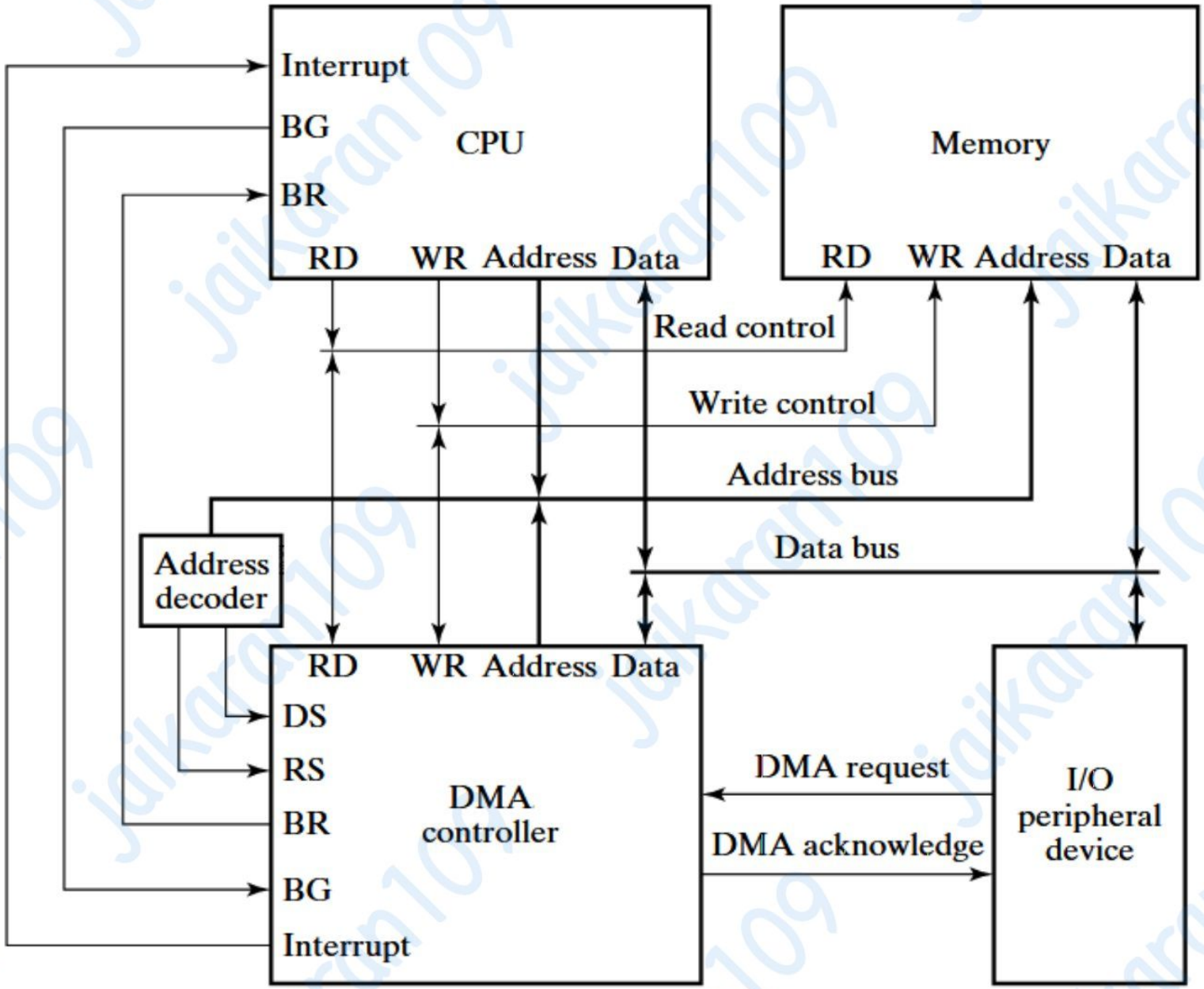
DMA Controller



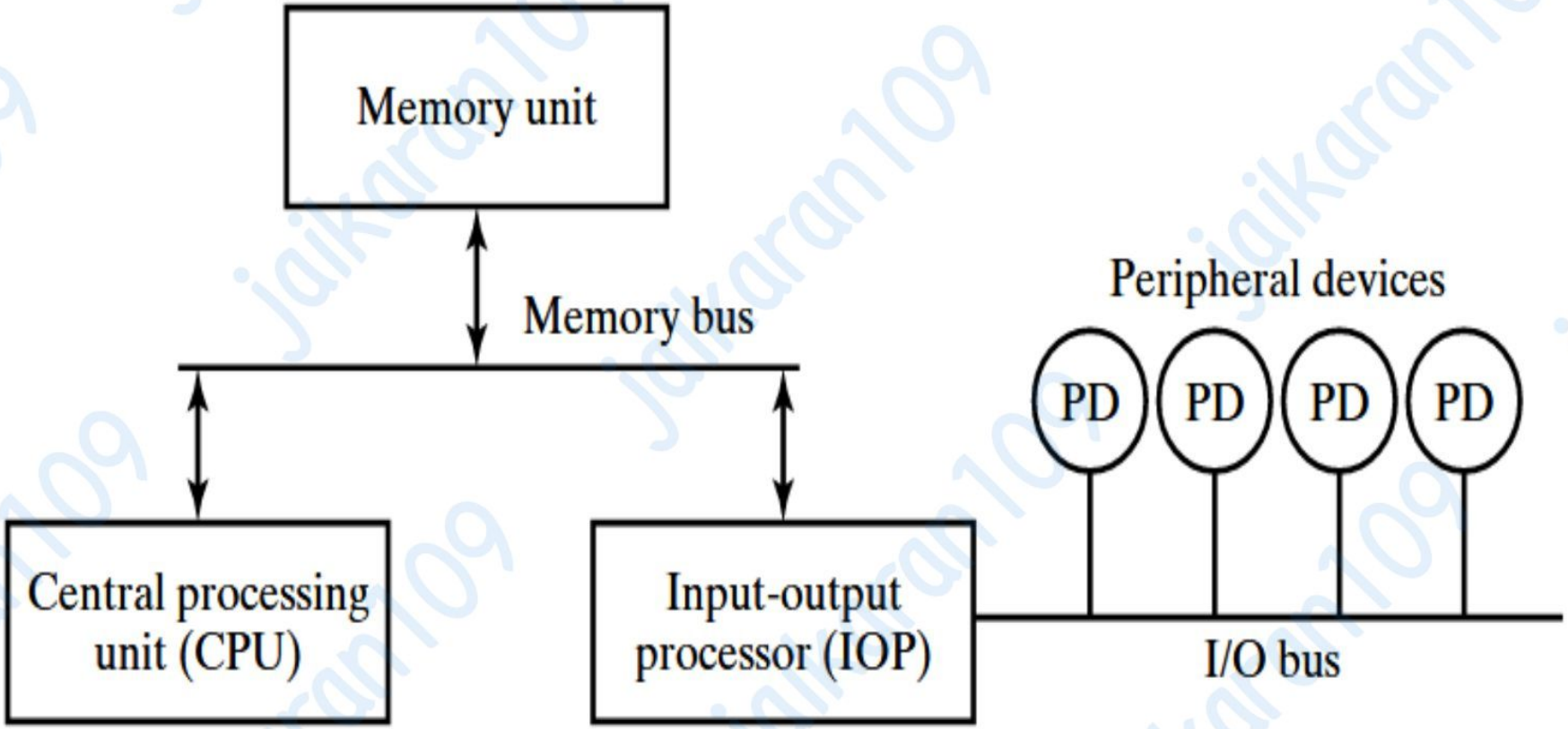
Initialization of DMA

- The CPU initializes the DMA by sending the following information through the data bus.
  - The starting address of the memory block where data are available(for read) or where data are to be stored(for write).
  - The word count, which is the number of words in the memory block.
  - Control to specify the mode of transfer such as read or write.
  - A control to start the DMA transfer.

DMA Transfer in a computer



Input-Output Transfer (IOP)

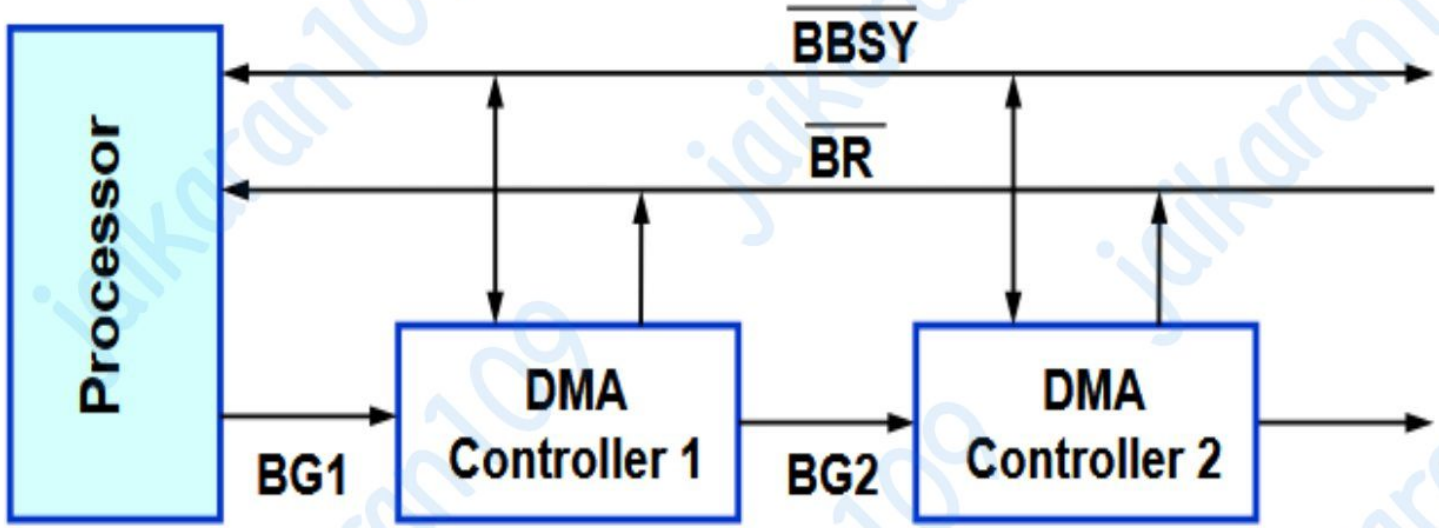


Input-Output Transfer (IOP)

- An IOP takes care of input and output tasks.
- IOP is similar to CPU except IOP can fetch and execute an I/O instruction.
- CPU is usually assigned task of initiating the I/O program and thus IOP.
- The CPU is assigned the task of initiating all operations, but I/O instructions are executed in IOP.
- When an I/O operation is required, the CPU informs the IOP where to find the I/O program and then leaves the transfer details to the IOP.
- CPU instructions also test I/O status conditions needed for making decisions on various I/O activities.
- IOP is also responsible of taking care of data synchronization, formats etc between CPU and I/O devices.
- In most computers CPU is master and IOP is slave.
- The IOP typically asks for CPU attention by means of Interrupt.

Bus Arbitration

- The device that is allowed to initiate data transfers on the bus at any given time is called Bus master.
- Bus arbitration is the process by which the next device becomes Bus master and will do the data transfer.
- Two approaches: Centralized Arbitration and Distributed Arbitration.



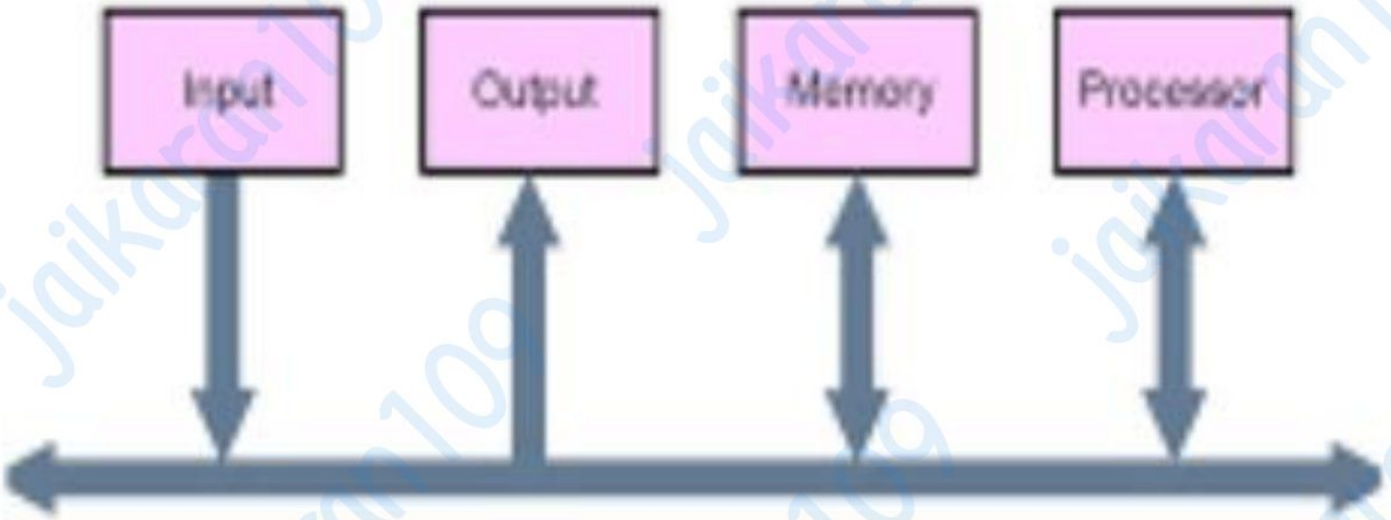
Types of Interrupt

- Major types of Interrupt are:
  - External Interrupt: It comes from I/O devices, from timing device, or from any other external source.
  - Internal Interrupt: It includes register overflow, invalid operation code, stack overflow etc.
  - Software Interrupt or Hardware Interrupt: External and Internal interrupts are initiated from signals that occur in the hardware of the CPU. *A software interrupt is initiated by executing an instruction.*
  - Priority Interrupt: In case several sources will request service simultaneously, in this case system must decide which device to service first. For ex: Polling, Daisy Chain.

Buses

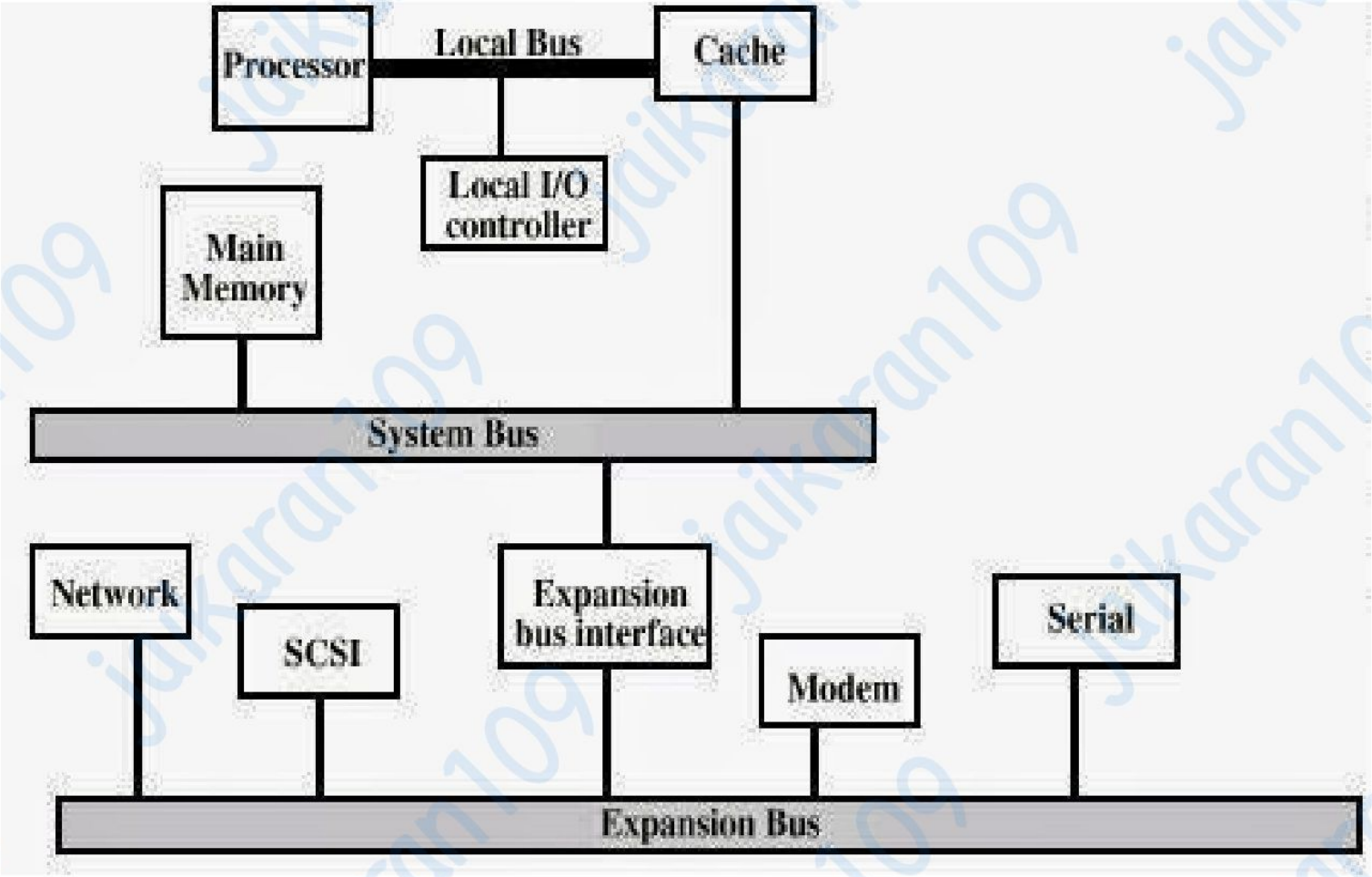
- Data Bus: Bi-directional and transfers data.
- Address Bus: Uni-directional and sends the address.
- Control Bus: R/W, BR,BG etc.

- Bus Structure: Single bus



Buses

- Bus Structure: Multi bus



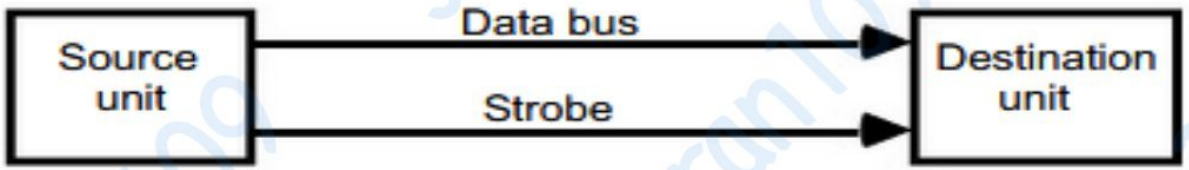
Synchronous or Asynchronous Data Transfer

- Internal operations in a digital system are synchronized by means of clock pulses supplied by common pulse generator.
- If the registers in the interface share a common clock with the CPU registers, the transfer is synchronous.
- Asynchronous data transfer requires that control signals be transmitted between communicating units to indicate the time at which data is being transmitted.
- Asynchronous data transfer can be accomplished by Strobe & Handshaking.

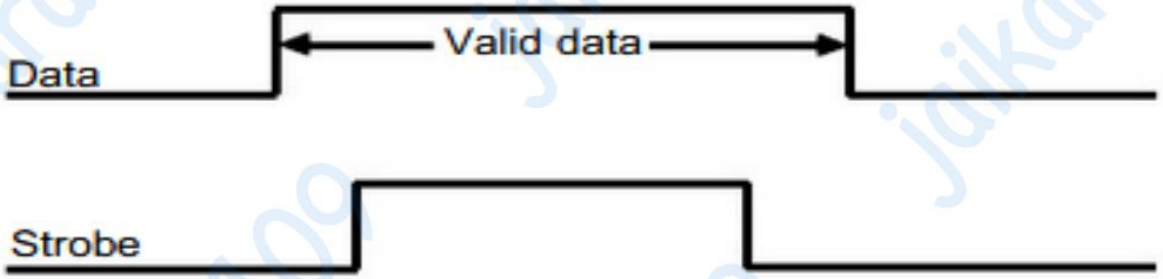
Strobe Control

- Strobe control:
  - It employs single control line.
  - It can be activated either by source or destination unit.

Block Diagram

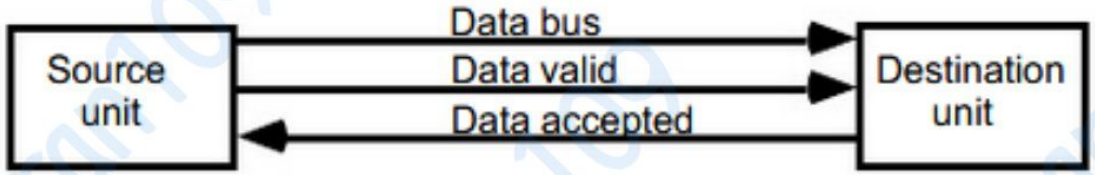


Timing Diagram

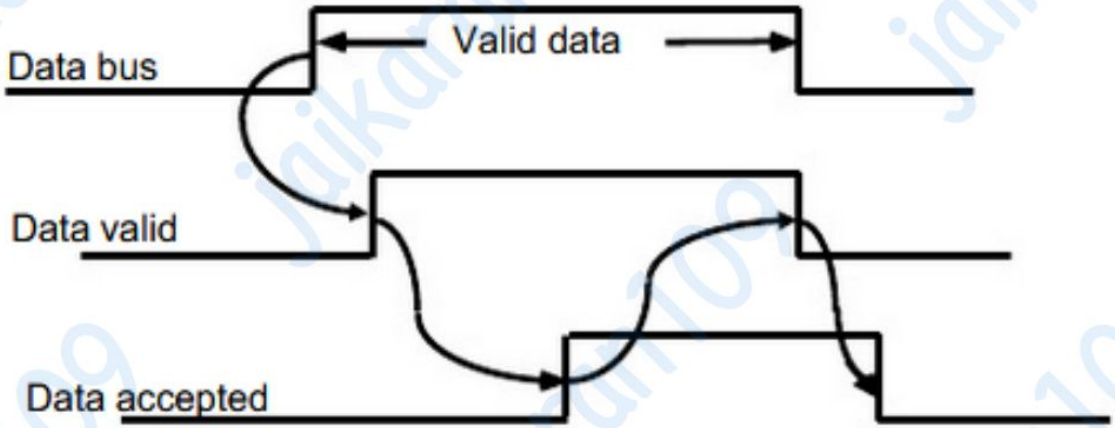


Handshaking

Block Diagram



Timing Diagram



Sequence of Events